

Dossier de sécurité

AvecStudy — audit interne du 23 septembre 2026

Audit interne. Ce dossier a été produit par l'éditeur sur son propre produit, à partir de référentiels publics. Il ne constitue ni une certification, ni une homologation, ni un test d'intrusion indépendant.

Nouh Tifouti — entrepreneur individuel — AvecStudy
SIREN 979 992 443 · 1 avenue d'Alsace, 90000 Belfort
Document généré le 2026-10-08

Dossier de sécurité AvecStudy

Audit interne du 23 septembre 2026. Dix constats, dix corrections vérifiées en production, huit contrôles permanents nouveaux — et trois points que cet audit n'a pas pu établir, qui portent la mention **NON VÉRIFIÉ** plutôt que « conforme ».

Sur la numérotation. Le plan demandé fixait deux bornes — 01-resume-direction et 16-risques-residuels . Les titres intermédiaires ont été composés pour couvrir l'ensemble des sujets demandés ; la correspondance figure ci-dessous. Renommer un fichier est sans conséquence : les renvois internes se font par nom, et il y en a peu.

État d'application

Les dix constats sont corrigés et vérifiés en production. Les migrations 0043 et 0044 ont été appliquées le 23 septembre 2026, après une sauvegarde logique vérifiée, et chaque fermeture est établie par un contrôle qui échouait avant.

La preuve la plus nette : cinq fonctions de la base répondaient **HTTP 200** à un appel anonyme — dont session_mfa_verifiee , le prédicat qui décide si un second facteur a été vérifié. Elles répondent toutes **401**.

Ce qui reste ouvert, et qui n'est pas de l'ordre du défaut mais de l'ordre du non-vérifié : la configuration des consoles Vercel et Supabase, et la séparation des environnements. Elles portent la mention **NON VÉRIFIÉ** tant que personne ne les a ouvertes.

Le détail figure en tête de 13-constats-et-corrections.md.

Par où commencer

Vous êtes	Lisez, dans cet ordre
Chef d'établissement	01 , puis 16
Réfèrent numérique	13 , 14 , 09 , 06
Délégué à la protection des données	07 , 08 , l'annexe de sous-traitance, puis 16
Pressé	01 seul — trois pages

Les documents

#	Document	Ce qu'il traite
01	<u>Résumé pour la direction</u>	Trois pages sans jargon. Le constat principal, les neuf autres, et le risque à peser.
02	<u>Périmètre, méthode, et ce qui n'a pas été vérifié</u>	Ce qui a été audité, comment, et les trois trous assumés.
03	<u>Architecture et circulation des données</u>	Ce que le système est. Qui voit quoi. Le cycle d'une copie.
04	<u>Modèle de menace</u>	Qui voudrait quoi, avec quels moyens. L'éditeur y figure.
05	<u>Authentification, sessions, abus</u>	Connexion, non-divulgateion, limitation des tentatives, second facteur.
06	<u>Autorisation et cloisonnement</u>	RLS, les deux schémas, les 202 tests, et la leçon la plus coûteuse.
07	<u>Données personnelles</u>	Ce qui est réellement collecté, relevé colonne par colonne. Durées, droits, AIPD.
08	<u>Hébergement et transferts hors EEE</u>	Où le calcul a lieu, où la base est, et ce qui reste à établir sur pièces.
09	<u>Sécurité web</u>	OWASP, en-têtes, CSRF, dépôt de fichiers.
10	<u>Journalisation et alertes</u>	Ce qui est journalisé, ce qui ne l'est jamais, et ce qui n'alerte personne.
11	<u>Sauvegardes, restauration, continuité, réversibilité</u>	Et si tout s'arrête.
12	<u>Incidents : procédure et exercices sur table</u>	Six étapes, dix scénarios joués sur le papier.
13	<u>Constats et corrections</u>	Le cœur factuel. Dix constats, leurs preuves, leurs correctifs.
14	<u>Les contrôles permanents</u>	Ce qui rejoue chaque constat, et comment le lancer.
15	<u>Questionnaire d'établissement, rempli</u>	Les questions qu'on nous pose, avec les « non » écrits en toutes lettres.
16	<u>Risques résiduels</u>	Ce qui reste après les corrections. Dix risques, dont l'éditeur lui-même.
—	<u>Annexe — contrôles manuels à faire en console</u>	Ce que l'audit n'a pas pu vérifier, et ce qu'il faut regarder exactement.
—	<u>Annexe — projet d'accord de sous-traitance</u>	Projet à valider par le DPO et/ou un juriste.

Correspondance avec les sujets demandés

Sujet	Où
Validation des dépôts de fichiers	09
Tests web (OWASP)	09
Nomenclature logicielle (SBOM)	09
Consoles Vercel et Supabase	02 et l'annexe des contrôles manuels — NON VÉRIFIÉ
Transferts hors EEE	08, 16 (R-02)
Conservation et effacement	07, 13 (F-09)
Droits des personnes, export élève	07, 16 (R-08)
Nécessité d'une AIPD	07
Journalisation	10
Alertes	10, 16 (R-09)
Procédure d'incident et exercices	12
Sauvegarde et restauration	11
Continuité et réversibilité	11
Disponibilité	11
Séparation des environnements	02 et l'annexe des contrôles manuels — NON VÉRIFIÉ
Sécurité du développement	14
Sécurité d'exploitation	10, 11, 12
Protection des mineurs	07
Inventaire des cookies	05, 09
Sécurité commerciale	07, 15
Questionnaire d'établissement	15
Plan d'action de sécurité	ci-dessous
Classification des constats	13

Plan d'action

Par ordre de priorité. Les quatre premiers points ne demandent que des accès.

#	Action	Qui	Lié à
1	Ouvrir les consoles Vercel et Supabase — la liste exacte est dans l' <u>annexe des contrôles manuels</u>	Éditeur	R-03, R-03 bis
2	Obtenir les accords de traitement des deux fournisseurs et les annexer	Éditeur + DPO	R-02
3	Jouer l'exercice de restauration de bout en bout, dans un projet jetable, et joindre le compte rendu	Éditeur	ch. 11
4	Vérifier qu'un déploiement d'aperçu ne pointe pas vers la base de production	Éditeur	R-03
5	Brancher une sonde de disponibilité externe et une alerte sur le bilan de la tâche planifiée	Éditeur	R-09
6	Fixer une durée de conservation du journal d'audit, et l'appliquer mécaniquement	DPO + Éditeur	R-07
7	Construire l'export complet des données d'un élève en une commande	Éditeur	R-08
8	Permettre à un utilisateur de révoquer ses autres sessions	Éditeur	ch. 12, scénario 1
9	Inscrire au contrat la réversibilité et l'export de fin d'année	Établissement + Éditeur	R-01
10	Conduire ou écarter l'AIPD	DPO de l'établissement	ch. 07

Ce que ce dossier n'écrit jamais

« Sécurité totale », « inviolable », « conforme ANSSI », « certifié RGPD », « certifié ISO 27001 », « hébergement souverain », « aucun risque ».

Aucune de ces phrases ne serait vraie. AvecStudy ne détient aucune certification, aucune qualification, aucune homologation, et n'a fait l'objet d'aucun test d'intrusion indépendant. Un établissement public mérite qu'on lui dise ce qui est plutôt que ce qui rassure.

01 — Résumé pour la direction

Destiné au chef d'établissement, au référent numérique et au délégué à la protection des données. Trois pages, sans jargon. Les pièces sont dans les documents suivants.

En une phrase

AvecStudy a été audité de fond en comble par son éditeur le 23 septembre 2026 : dix défauts ont été trouvés, **dix ont été corrigés et vérifiés sur le service en ligne**, et chacun a reçu un test automatique qui empêchera son retour — mais AvecStudy est édité par une seule personne, et c'est le risque que ce dossier vous demande de peser en premier.

Deux réglages d'hébergement n'ont pas pu être contrôlés faute d'accès aux consoles de nos fournisseurs pendant l'audit. Ils portent la mention **NON VÉRIFIÉ**, pas « conforme ».

Ce qu'AvecStudy fait de vos données

Les élèves et les professeurs se connectent avec un identifiant remis par l'établissement. Ils déposent des devoirs, des copies, des corrections, des documents de cours. AvecStudy ne demande jamais l'identifiant national élève (INE), ne diffuse aucune publicité, ne revend rien, et n'est jamais payé par les familles.

Chaque personne ne voit que ce qui la concerne, et cette règle n'est pas un réglage d'affichage : elle est appliquée **dans la base de données elle-même**, par 144 politiques que 202 tests automatiques vérifient à chaque livraison. Un professeur qui n'enseigne pas à une classe ne peut pas lire les copies de cette classe, même en contournant l'écran.

Le défaut le plus important que nous avons trouvé

Le calcul se faisait aux États-Unis.

La base de données était bien en Irlande — donc en Europe. Mais les pages étaient **fabriquées** sur un serveur situé à Washington. À chaque affichage, les noms des élèves, les intitulés de classe et le contenu des copies traversaient l'Atlantique.

Aucune région n'avait été choisie : l'hébergeur avait appliqué son réglage par défaut, et rien dans nos fichiers ne le contredisait. Personne ne s'en était aperçu parce qu'il n'y avait rien à relire — seulement une absence.

C'est corrigé : le calcul est désormais déclaré à Paris. Un test automatique le vérifie à chaque livraison et refuse toute région hors d'Europe.

Ce que nous ne pouvons pas encore vous affirmer : que plus aucune donnée ne quitte l'Europe. Une région d'hébergement est un réglage technique, pas un engagement contractuel. Les sauvegardes, les journaux techniques et l'assistance de nos deux fournisseurs relèvent de leurs propres contrats, qui restent à examiner pièce par pièce. Le document `08-hebergement-et-transferts.md` dit exactement ce qui est établi et ce qui ne l'est pas.

Les neuf autres défauts, en clair

	Ce qui n'allait pas	Ce que cela permettait	Corrigé
F-01	Cinq fonctions internes de la base restaient appelables sans aucun compte	Obtenir une référence d'accusé de remise sans être connecté — pas une copie, pas un nom. Et interroger le mécanisme qui décide si un second facteur a été vérifié	oui
F-07	Le ralentissement des tentatives de connexion visait chaque compte séparément	Essayer trois mots de passe sur huit cents comptes sans jamais être ralenti	oui
F-06	Une protection contre les formulaires piégés était écrite mais pas branchée	Rien d'exploitable avec un navigateur récent, mais la protection n'était pas celle qu'on croyait	oui
F-02	Un compte suspendu gardait ses droits tant que sa session vivait	Rien en pratique — la suspension coupait déjà les sessions — mais la garantie reposait sur une habitude	oui
F-09	Une durée de conservation de 24 h était écrite et n'était appliquée nulle part	Trois lignes techniques conservées trois jours au lieu d'un	oui
F-10	Aucune adresse normalisée pour signaler une faille	Un référent numérique n'avait pas de porte où frapper	oui
F-03 / F-04	Deux tables techniques dérogeaient à la règle commune	Rien : elles n'étaient atteignables par personne	oui
F-05	Une règle de sécurité du navigateur est assouplie sur douze pages publiques	Rien : ces pages n'affichent aucun texte saisi par quiconque	assumé et justifié

Ce que nous avons vérifié et qui va bien

- **Aucun mot de passe, aucune clé, aucun secret** dans les 100 versions successives du code, ni dans ce que le navigateur reçoit (6 pages et 13 fichiers de code inspectés intégralement).
- **Aucune faille connue** dans les bibliothèques utilisées, toutes sévérités confondues.
- **Aucun moyen de se faire passer pour quelqu'un d'autre** : les 46 fonctions internes qui acceptent l'identité d'une personne en paramètre sont réservées au serveur, et aucune n'est atteignable depuis un navigateur.
- **Les fichiers déposés sont reconnus sur leur contenu**, pas sur leur nom : un fichier appelé `cours.pdf` qui n'en est pas un est refusé. Rien n'est ouvert dans le navigateur : tout se télécharge.
- **Les journaux ne contiennent jamais** un mot de passe, un jeton, un cookie, une copie ni un nom d'élève — seulement un code d'erreur et un contexte.

Ce que nous ne pouvons pas vous dire

Trois choses n'ont pas pu être vérifiées faute d'accès aux consoles d'administration de nos deux fournisseurs pendant cet audit : leur configuration exacte, et la séparation stricte entre l'environnement de test et celui de production. Ces trois lignes portent la mention **NON VÉRIFIÉ**, et non « conforme ». Elles sont détaillées dans `02-perimetre-et-methode.md`.

Le risque que nous vous demandons de peser

AvecStudy est édité par **une entreprise individuelle — une seule personne**.

Cela veut dire, concrètement, et il faut que ce soit écrit noir sur blanc :

- il n'y a pas d'équipe de sécurité de garde, pas d'astreinte la nuit, pas de délai de réponse contractuel ;
- une indisponibilité de cette personne — maladie, accident — est une indisponibilité du support ;
- l'audit que vous lisez a été mené par celui-là même qui a écrit le code.

Ce qui existe en face, et qui est vérifiable : le code est intégralement versionné, la base est restaurable, les données sont exportables, et 202 tests automatiques passent avant chaque livraison. Le document `11-sauvegards-et-continuite.md` décrit ce qu'un établissement peut récupérer et comment, si l'éditeur disparaissait.

Nous n'écrirons pas que le service est « sans risque », ni « inviolable », ni « conforme ANSSI », ni « certifié RGPD ». Aucune de ces phrases ne serait vraie, et un établissement public mérite qu'on lui dise ce qui est plutôt que ce qui rassure.

Où aller ensuite

Vous êtes	Lisez d'abord
Chef d'établissement	ce document, puis <code>16-risques-residuels.md</code>
Référent numérique	<code>13-constats-et-corrections.md</code> et <code>14-contrôles-permanents.md</code>
Délégué à la protection des données	<code>07-donnees-personnelles-et-rgpd.md</code> , <code>08-hebergement-et-transferts.md</code> , et l'annexe de sous-traitance
Toute personne pressée	le tableau ci-dessus, et la section « le risque que nous vous demandons de peser »

02 — Périmètre, méthode, et ce qui n'a pas été vérifié

Quatre choses différentes, qu'on confond souvent

Ces quatre mots reviennent dans les appels d'offres et dans les questionnaires d'établissement, et ils ne désignent pas la même chose. Ce dossier appartient à la première catégorie, et à elle seule.

	Ce que c'est	Qui l'établit	AvecStudy
Audit interne	Un examen du produit par ceux qui le font, appuyé sur des référentiels publics. Sa valeur tient à ses preuves reproductibles, pas à la qualité de son auteur.	L'éditeur lui-même	C'est ce document.
Audit indépendant	Le même travail, mené par un tiers qui n'a pas écrit le code et n'a rien à défendre. Test d'intrusion, revue de code, audit d'architecture.	Un prestataire extérieur, mandaté	Aucun. Personne d'extérieur n'a relu ce travail.
Certification	Une attestation délivrée par un organisme accrédité, après examen, selon un référentiel normalisé — ISO 27001, SecNumCloud, HDS, label CNIL. Elle se vérifie dans un registre public.	Un organisme certificateur	Aucune. Ni ISO 27001, ni SecNumCloud, ni HDS, ni label RGPD.
Conformité réglementaire	Le respect d'une obligation légale — le RGPD, par exemple. Elle ne se « certifie » pas : elle se démontre, en continu, et c'est le responsable de traitement qui en répond.	L'établissement, avec son DPO	Ce dossier fournit des pièces techniques. Il ne conclut pas à la conformité, et il n'en a pas le pouvoir.

Trois conséquences pratiques, qu'il vaut mieux lire maintenant que découvrir plus tard :

- Personne ne peut écrire « AvecStudy est certifié RGPD » : **cette certification n'existe pas**, pour aucun éditeur.
- « Conforme ANSSI » ne veut rien dire sans qualification formelle. Cet audit s'appuie sur les recommandations publiques de l'ANSSI, ce qui est autre chose, et ne donne droit à aucune mention.
- La conformité du **traitement** est celle de l'établissement. Un dossier de sous-traitant, aussi fourni soit-il, ne la remplace pas — il l'alimente.

Ce que ce dossier est, et ce qu'il n'est pas

C'est un **audit interne approfondi**, mené par l'éditeur sur son propre produit, appuyé sur des référentiels publics : OWASP ASVS et le Top 10, les recommandations d'hygiène de l'ANSSI, le RGPD et les référentiels de la CNIL.

Cette distinction n'est pas une formule de prudence. Un audit mené par celui qui a écrit le code trouve bien les défauts d'inattention — cet audit en a trouvé dix — et trouve mal les défauts de conception, parce qu'il les partage. Ce que ce dossier vaut, il le vaut par ses preuves reproductibles, pas par la qualité de son auteur.

Ce qui a été audité

- Le dépôt Git d'AvecStudy, dans son intégralité et sur ses 93 commits.
- La production, `https://avecstudy.fr`.
- Le projet Supabase d'AvecStudy, au niveau SQL.
- Les déploiements Vercel d'AvecStudy, tels qu'ils répondent sur le réseau.
- Le stockage utilisé par AvecStudy.
- Les comptes synthétiques créés par `recette:complete`.

Ce qui n'a pas été touché, et ne devait pas l'être

Aucun système de l'Éducation nationale. Ni ÉCLAT-BFC, ni Pronote, ni EduConnect, ni le site d'un lycée. Aucun tiers contacté. Aucun déni de service, aucune saturation d'API. Aucune donnée réelle d'élève. Aucune tentative de persistance. Aucun fichier sorti des environnements autorisés, pas même synthétique.

Cette dernière interdiction a eu un effet concret sur un constat : **F-07**, le balayage d'établissement, n'a pas été reproduit offensivement. Pulvériser des mots de passe sur des comptes, même synthétiques, revient à saturer une API. Le constat repose donc sur une preuve structurelle — la signature de `auth_compter_echecs` ne prend qu'un profil — et non sur une démonstration. Il est dit comme tel dans `13-constats-et-corrections.md`.

Comment les tests offensifs ont été menés

Limités, reproductibles, non destructifs.

Concrètement : la vérification de la barrière d'origine (F-06) tient en **six requêtes HTTP**, vers une page publique, sans déclencher la moindre action serveur. La vérification de la fuite de secrets (§16) lit six pages et treize scripts. La reproduction de F-01 est **un** appel. Rien de ce qui a été joué n'aurait été perceptible dans un journal d'accès comme autre chose qu'un visiteur.

Chaque test offensif qui a trouvé quelque chose est devenu un **contrôle permanent** : il échouait avant la correction, il doit passer après. Un test qui n'a jamais échoué ne prouve pas grand-chose.

Les trois manières dont une vérification a été faite

Elles ne se valent pas, et le dossier les distingue partout.

Joué. Une requête a été envoyée, une page ouverte, une requête SQL exécutée, et le résultat est retranscrit. C'est le seul niveau qui vaut preuve.

Structurel. Le code ou le schéma ne permet pas la chose, et on le montre — une signature de fonction, une politique RLS, une liste de droits. C'est solide quand la lecture est mécanique et non interprétative, et cet audit a rappelé pourquoi il faut s'en méfier autrement : sept tests verts couvraient `verifierMutation`, et **personne ne l'appelait**.

Non vérifié. Personne ne l'a joué. La ligne le dit.

Ce qui n'a pas pu être vérifié

Trois points, et il faut les lire comme des trous, pas comme des réserves de style.

§18 — La configuration de la console Vercel

NON VÉRIFIÉ. Aucun identifiant Vercel n'était disponible dans l'environnement d'audit : ni `VERCEL_TOKEN`, ni session de CLI, ni répertoire `.vercel` dans le dépôt.

Ce qui reste à établir, et qui demande une session connectée :

- qui a accès au projet, avec quel rôle, et si le second facteur est exigé sur chaque compte ;
- quelles variables d'environnement existent, sur quels environnements, et lesquelles sont marquées sensibles ;
- si la protection des déploiements d'aperçu est active ;
- quels journaux sont conservés, et combien de temps.

§19 — La configuration de la console Supabase

NON VÉRIFIÉ. L'audit disposait d'une chaîne de connexion à la base, ce qui a permis de tout vérifier au niveau SQL — tables, politiques, fonctions, droits, volumes. Cela ne dit rien des réglages de **projet** :

- la sauvegarde ponctuelle (PITR) : activée ou non, profondeur ;
- les restrictions réseau sur la base ;
- le second facteur sur le compte propriétaire du projet ;
- la rotation et la date des clés de service ;
- la liste des sous-traitants ultérieurs déclarés par le fournisseur.

§31 — La séparation des environnements

NON VÉRIFIÉ. Établir qu'un déploiement d'aperçu ne pointe pas vers la base de production demande de lire les variables d'environnement par environnement, dans la console. C'est exactement le genre de point qu'on ne peut pas déduire du dépôt : le code lit `SUPABASE_URL`, il ne décide pas de sa valeur.

Ce que le dépôt montre en revanche, et qui est vérifié : le code **prévoit** la distinction (`APP_ENV`, `VERCEL_ENV`), et l'exception d'origine accordée aux déploiements d'aperçu est étroite — elle n'accepte que l'origine du déploiement lui-même et disparaît en production.

Ce qui a été laissé délibérément en l'état

Aucune protection permanente n'a été désactivée pour faciliter l'audit. La production a gardé sa CSP, ses entêtes, sa limitation de tentatives et ses politiques RLS pendant toute la durée des vérifications.

Ce que ce dossier ne contient pas

Aucun secret, même partiel, même expiré. Aucun exploit réutilisable : les constats disent le résultat obtenu, pas la manœuvre complète. Aucune donnée personnelle. Aucune URL signée ni adresse de ressource sensible.

Les contrôles permanents suivent la même règle dans leur propre sortie : `verifier:fuites` cherche des valeurs de secrets et n'en imprime jamais une, pas même tronquée — il dit le **nom** de la variable et le fichier. Un contrôle de fuite qui imprime ce qu'il a trouvé recopie la fuite dans les journaux de la recette, et les journaux voyagent plus loin que la page.

Un piège de méthode, rencontré à la fin de l'audit

Le balayage de l'historique Git cherche des **formes** de secret : l'en-tête d'un jeton JWT, le préfixe d'une clé secrète, une URL de base portant un mot de passe.

Relancé après les corrections, il a signalé sept correspondances là où il n'en trouvait aucune au départ. Vérification faite, les sept étaient dans un seul fichier — `scripts/recette/fuites.mjs` — et c'étaient **les motifs de détection eux-mêmes**, écrits dans les commentaires et les expressions régulières du détecteur de fuites ajouté par cet audit.

Aucun secret, donc. Mais il faut le dire pour deux raisons.

D'abord parce qu'un compteur qui passe de zéro à sept sans explication est précisément ce qui use un contrôle : au bout de quelques fausses alertes, personne ne le relit. Le compteur `secrets_dans_git` vaut toujours **0**, et la ligne brute vaut 7 — les deux sont vraies, et il faut savoir pourquoi.

Ensuite parce que c'est le genre de vérification qu'on est tenté d'écarter d'un geste : « ah, c'est sûrement mon propre scanner ». C'est sûrement le cas neuf fois sur dix, et la dixième est un vrai secret. Chaque correspondance a donc été ouverte et lue, une par une, avant d'être écartée.

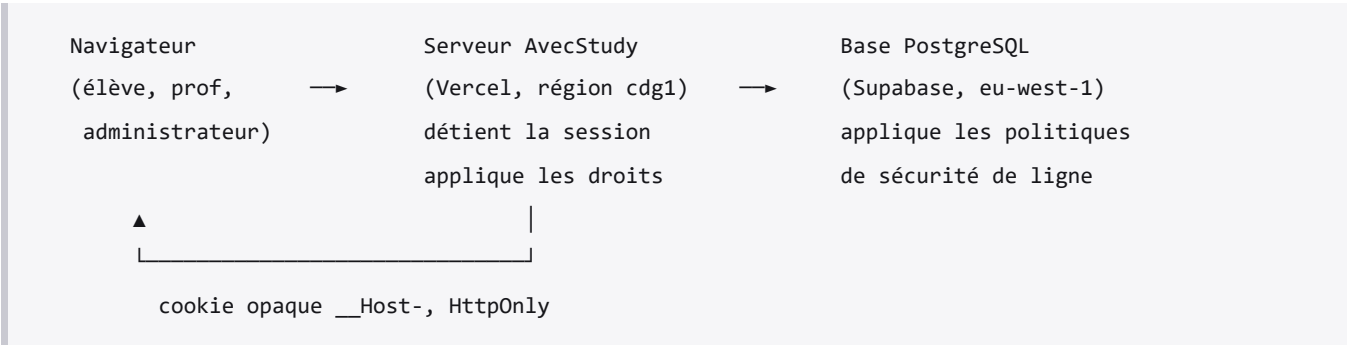
Dates et point de départ

Audit mené le 23 septembre 2026. Point de départ : commit `ed243bc`, arbre propre, `main` et `refonte-v2` alignés sur leurs branches distantes. Relevé du schéma avant travaux : 68 tables, 144 politiques, 113 fonctions, 52 déclencheurs, 5 extensions, 14 rôles.

03 — Architecture et circulation des données

Ce que le système est, en une page, pour qu'un référent numérique puisse juger le reste du dossier.

Le principe : le navigateur ne parle jamais à la base



C'est le patron dit **BFF** (*backend for frontend*). Sa conséquence la plus importante : **aucune clé de base de données n'atteint le navigateur**. Le navigateur ne détient qu'un cookie sans contenu, qui n'est qu'une clé vers une ligne de session.

Vérifié, et pas seulement affirmé : `npm run verifier:fuites` inspecte intégralement le HTML servi, la charge utile React qu'il contient et chacun des fichiers de code chargés — 6 pages et 13 scripts au 23 septembre 2026. Aucun secret, aucune forme de secret.

Les composants

Composant	Rôle	Fournisseur
Application web	Rendu des pages, actions serveur, session	Vercel, région <code>cdg1</code> (Paris)
Base de données	Toutes les données, et l'application des droits	Supabase / AWS <code>eu-west-1</code> (Irlande)
Authentification	Vérification des mots de passe, second facteur TOTP	Supabase Auth
Stockage de fichiers	Copies déposées, supports de cours	Supabase Storage
File de travaux	Conversion de documents, traitements différés	Table en base, drainée par une tâche planifiée

Aucun autre service ne reçoit de donnée. Pas d'outil de mesure d'audience, pas de service de surveillance d'erreurs tiers, pas de police de caractères distante — elles sont auto-hébergées. La politique de sécurité du contenu (`connect-src 'self'`, `font-src 'self'`, `img-src 'self'`) l'interdit techniquement à une page.

Deux schémas, une frontière

	study	study_prive
Exposé par l'API	oui	jamais
Droits pour anon	aucun	aucun
Droits pour authenticated	par politique de ligne	aucun
Contenu	travail scolaire, identités, structure scolaire	sessions, alias techniques, tentatives de connexion, file de travaux

La frontière est vérifiée à chaque recette, directement en base, et non déduite d'une configuration.

Qui voit quoi

	Voit	Ne voit pas
Élève	Ses devoirs, ses remises, ses corrections, les séances de ses classes, l'entraide de ses groupes	Le travail des autres élèves, les brouillons de ses professeurs, tout autre établissement
Professeur	Les séances et copies des espaces qu'il enseigne	Les classes qu'il n'enseigne pas — et il perd l'accès dès que son affectation est retirée
Administrateur d'établissement	La structure, les comptes, les accès, les signalements de son lycée	Le contenu du travail scolaire des élèves
Éditeur (exploitant)	Les établissements, le commercial, l'administration	Le travail des élèves lui est fermé — il ne peut ni le lire ni y écrire, et quatre tests de la batterie le vérifient

Cette dernière ligne est le point qu'un établissement doit retenir : l'éditeur d'AvecStudy n'a pas accès aux copies des élèves par les chemins applicatifs.

Réserve d'honnêteté. Il détient en revanche les accès à l'infrastructure, et donc à la base. Aucune architecture ne peut prétendre le contraire : celui qui exploite une base peut la lire. Ce qui existe en face, c'est la traçabilité des actes applicatifs, le fait que chaque contournement de RLS passe par un motif déclaré et relisible, et — surtout — le fait que ce soit écrit ici.

Le cycle d'une copie

1. L'élève dépose. La ligne naît **réservée**, jamais servable.
2. Le fichier part au stockage, sous un chemin fabriqué côté serveur : trois identifiants, sans extension, jamais le nom d'origine.
3. Le type est reconnu **sur les octets**. Un fichier nommé `.pdf` qui n'en est pas un est refusé.
4. Le worker déclare le fichier disponible — lui seul le peut.
5. L'élève reçoit une **référence d'accusé de remise**, consultable et imprimable.

6. La version est **scellée** : elle ne change plus. Une nouvelle remise crée une nouvelle version, et l'ancienne cesse d'être servie tout en restant à l'historique.
7. Le professeur affecté peut l'ouvrir. Personne d'autre.

Chacune de ces étapes est couverte par au moins un test de la batterie RLS.

Ce qui circule hors d'Europe

Après la correction du constat F-08 : rien, au niveau des régions déclarées. Le calcul est à Paris, la base en Irlande.

Ce qui n'est pas établi : la localisation des sauvegardes, des journaux des fournisseurs et des accès de leur support. Voir `08-hebergement-et-transferts.md` et le risque résiduel R-02.

Pour aller plus loin

Le dictionnaire de données complet — chaque table, chaque colonne, chaque politique — est dans `docs/03-dictionnaire-de-donnees.md`, hors de ce dossier de sécurité. Il fait 2 479 lignes et décrit l'état réel du schéma.

04 — Modèle de menace

Qui voudrait quoi, avec quels moyens, et ce que le système oppose. Un modèle de menace qui liste des attaquants abstraits ne sert à rien : celui-ci part de ce qui arrive réellement dans un lycée.

Ce qu'il y a à prendre

Par ordre de valeur pour un attaquant, et d'impact pour un élève :

1. **Le contenu d'une copie et son appréciation.** Ce n'est pas une donnée sensible au sens du RGPD, mais c'est ce qui humilie un adolescent si ça circule dans un groupe de classe.
2. **Les identifiants d'un compte**, surtout d'un professeur ou d'un administrateur — ils ouvrent les copies des autres.
3. **Les listes nominatives** : qui est dans quelle classe.
4. **La disponibilité** elle-même : empêcher une remise la veille d'une échéance.

Les attaquants réalistes

A — L'élève curieux ou malveillant

De loin le plus probable. Il est déjà authentifié, il connaît les noms de ses camarades, il a du temps, et il n'a pas besoin de compétences : il suffit souvent de changer un identifiant dans une adresse.

Ce qu'il tentera : ouvrir la copie d'un autre, lire un devoir avant publication, voir une correction avant sa diffusion, deviner la référence d'accusé d'un camarade, usurper un compte laissé ouvert.

Ce qui l'arrête : l'autorisation vit dans la base, pas dans l'écran. Changer un identifiant dans une adresse ne sert à rien : la politique de ligne ne rend pas la ligne. 202 tests le vérifient sur du vrai PostgreSQL.

Ce que l'audit a trouvé sur cet axe : F-01 — la référence d'accusé de remise était calculable par un appel anonyme. Il fallait encore connaître l'identifiant de version, que RLS protège, mais la promesse « non devinable » était affaiblie. Corrigé.

B — Le camarade avec un accès physique au poste

Un poste du CDI, une session laissée ouverte, un téléphone non verrouillé.

Ce qui l'arrête : la session « poste partagé » expire beaucoup plus vite ; le cookie est `HttpOnly` et opaque, donc ni lisible par un script ni exploitable ailleurs ; un compte administratif prend toujours la fenêtre la plus courte, même sur un appareil personnel.

Ce qui manque : un utilisateur ne peut pas révoquer lui-même ses autres sessions. Relevé par l'exercice sur table n° 1.

C — L'attaquant opportuniste depuis Internet

Il ne vise pas AvecStudy en particulier. Il balaie, il essaie des mots de passe courants, il cherche une API ouverte.

Ce qui l'arrête : aucune fonction de base exécutable sans compte (depuis la correction de F-01) ; la limitation de tentatives ; l'absence d'énumération de comptes ; les en-têtes de sécurité ; la barrière d'origine sur toute mutation.

Ce que l'audit a trouvé : F-07 — le balayage d'établissement passait sous tous les compteurs. C'est le scénario de cet attaquant dans un lycée, où les mots de passe sont distribués à la rentrée et se ressemblent. Corrigé.

D — Le professeur ou l'administrateur dont le compte est compromis

Ce qui limite : sans second facteur, un administrateur **ne fait rien** de privilégié — il verrait ses écrans, il ne pourrait ni créer, ni suspendre, ni réinitialiser, ni modérer. Vérifié par les tests T16 et T16b.

Ce que l'audit a trouvé : F-02 — un compte suspendu gardait ses rôles tant que sa session vivait. Le risque était déjà fermé par la révocation des sessions, mais reposait sur une habitude. Rendu structurel.

E — L'éditeur lui-même

Il faut le nommer, parce qu'un modèle de menace qui omet l'exploitant est incomplet.

Ce qui limite, côté applicatif : le travail des élèves est fermé à l'exploitant — quatre tests de la batterie le vérifient, en lecture comme en écriture. Tout contournement de RLS passe par `clientExploitation(motif)`, dont le paramètre est un **type fermé** de huit motifs, chacun commenté et relisible dans un diff.

Ce qui ne limite pas : celui qui exploite une base peut la lire. Aucune architecture ne peut prétendre le contraire, et ce dossier ne le prétend pas. Ce qui existe en face, c'est la traçabilité des actes applicatifs et le fait que cette limite soit écrite ici plutôt que tue.

F — Le fournisseur d'hébergement

Vercel et Supabase peuvent techniquement accéder à l'infrastructure qu'ils opèrent. C'est le sens même de la sous-traitance, et cela se traite par le contrat, pas par le code.

Ce que l'audit a trouvé sur cet axe : F-08 — le rendu serveur s'exécutait aux États-Unis, sans que rien ne le déclare. Corrigé. Ce qui reste : la localisation contractuelle des sauvegardes, des journaux et des accès du support, **NON VÉRIFIÉE** (R-02).

Ce qui est hors du modèle

Ce que ce dossier ne prétend pas couvrir, et qu'il ne faut donc pas croire couvert :

- **Un poste d'élève ou de professeur déjà compromis.** Si un logiciel espion est installé sur la machine, AvecStudy n'y peut rien — il verra ce que la personne voit.
- **Un attaquant étatique ou une chaîne d'approvisionnement compromise.** Hors de portée d'une structure de cette taille, et le prétendre serait faux.

- **Le réseau de l'établissement**, l'ENT, Pronote, EduConnect. Hors périmètre, et le mandat interdisait de les toucher.
- **Le déni de service**. Aucune protection spécifique au-delà de ce que l'hébergeur applique par défaut.

La menace qui n'est pas un attaquant

Elle mérite sa propre section, parce que c'est celle qui s'est réalisée le plus souvent dans l'histoire de ce produit : **l'erreur silencieuse**.

Trois exemples réels, tous trouvés par des contrôles et aucun par un attaquant :

- une politique de sécurité du contenu qui empêchait **tous** les scripts de se charger en production — sans la moindre erreur côté serveur ;
- deux variables de couleur qui n'existaient pas : ni la compilation, ni le typage, ni les tests ne le voient — seulement l'écran ;
- une fonction de sécurité couverte par sept tests verts, que **personne n'appelait**.

C'est pourquoi les contrôles de ce dossier lisent **ce qui est servi**, et non ce qui est écrit. Et pourquoi chacun doit avoir été vu rouge avant d'être cru vert.

05 — Authentification, sessions, abus

Comment on se connecte

Un code d'établissement, un identifiant local remis par le lycée, un mot de passe. Pas d'adresse électronique pour les élèves, pas d'inscription libre, pas de fédération d'identité.

Derrière, l'identité technique passée au fournisseur d'authentification est un **alias opaque**. Un test vérifie qu'il ne contient ni nom ni classe : le fournisseur ne sait donc pas qui est qui.

L'ordre des opérations, et pourquoi il est ce qu'il est

C'est la partie du code où l'ordre **est** la sécurité.

1. **Résoudre l'identifiant.** S'il est inconnu, on vérifie quand même un secret contre une identité factice, puis on renvoie le refus générique. Sans cela, un identifiant inconnu répondrait plus vite qu'un identifiant réel, et le service deviendrait un annuaire.
2. **Appliquer la limitation de tentatives**, *avant* de parler au fournisseur. Autrement, le service se transforme en oracle de mots de passe : même bridé, il répondrait « ce mot de passe est bon » à volume illimité.
3. **Vérifier le secret.**
4. **Seulement ensuite**, regarder l'état du compte. Un compte suspendu et un mot de passe faux produisent exactement le même refus, avec le même vocabulaire.

Quatre tests d'unité vérifient qu'un identifiant inconnu, un mot de passe faux, un compte suspendu et une adhésion terminée sont **indiscernables** vus du navigateur.

Limitation des tentatives

Par compte

Cinq échecs en quinze minutes, puis une temporisation qui double : 30 s, 60 s, 120 s... plafonnée à quinze minutes. Une connexion réussie efface l'ardoise — sinon cinq erreurs de frappe en début de matinée puniraient encore l'élève à midi.

Pourquoi pas par adresse IP : huit cents élèves d'un lycée sortent par la même adresse publique. Bloquer sur l'IP reviendrait à fermer l'établissement à la première erreur de frappe de la salle 204.

En cas de panne de lecture du compteur, on suppose qu'il y a eu des échecs : mieux vaut ralentir une personne légitime que d'ouvrir un compte au forçage.

Par établissement — ajouté par cet audit

Le compteur par compte ne voit pas l'attaque la plus probable ici. Un pulvérisateur de mots de passe essaie trois mots de passe plausibles sur huit cents comptes : chaque compteur reste à trois, sous le seuil, et rien ne ralentit jamais.

Au-delà de **trente cibles distinctes** en échec dans un établissement sur quinze minutes, le seuil par compte descend de cinq à **deux**.

Trois précisions importantes :

- **Cibles distinctes**, pas tentatives : mille échecs sur un seul compte sont un compte oublié, et c'est l'autre compteur qui s'en occupe.
- **Rien ne se verrouille**. Une personne dont le mot de passe est bon entre, alerte ou pas — un test le vérifie explicitement. Un verrou déclenché par un tiers serait une arme qu'on lui tend : il suffirait d'un balayage volontaire un matin de rentrée pour empêcher un lycée entier de se connecter.
- **Le doute penche dans l'autre sens** que pour le compteur par compte : si la lecture échoue, on suppose qu'il n'y a pas de balayage. Supposer l'inverse transformerait une panne technique en gêne pour huit cents personnes.

Voir le constat F-07 : la donnée était collectée depuis la migration 0015 « pour repérer un balayage », et rien ne la relisait.

Ce que cela ne fait pas : prévenir quelqu'un. Le balayage est ralenti, il n'est pas signalé. Voir R-09.

Sessions

Le cookie est **opaque** : `__Host-avecstudy_session`, une valeur sans contenu, qui n'est qu'une clé vers une ligne de `study_prive.sessions`.

Attribut	Valeur	Effet
Préfixe <code>__Host-</code>		Le navigateur refuse le cookie s'il n'est pas servi en HTTPS depuis le domaine exact. Un sous-domaine compromis ne peut pas en poser un.
<code>HttpOnly</code>	oui	Inaccessible au JavaScript de la page.
<code>Secure</code>	en production	Jamais transmis en clair.
<code>SameSite</code>	<code>Lax</code>	Une requête partie d'un autre site n'emporte pas la session.
<code>path</code>	<code>/</code>	

Conséquences pratiques : un cookie volé ne révèle rien par lui-même, et il se **révoque** — ce qu'un jeton auto-porteur ne permet pas.

Durées

Deux échéances par session : une par **inactivité**, une **absolue**. Un compte portant un rôle administratif prend toujours la fenêtre la plus courte, même sur un appareil personnel. Un poste partagé expire beaucoup plus vite qu'un appareil personnel.

Relecture à chaque requête

La session est relue en base à chaque requête. C'est ce qui permet qu'une habilitation retirée ferme l'espace à **la requête suivante**, sans attendre une expiration.

L'audit y a trouvé un défaut (F-02) : la lecture prenait les rôles de l'adhésion sans regarder son état. Une session vivante d'un compte suspendu gardait donc ses écrans. Le risque était déjà fermé — les deux chemins de suspension révoquent les sessions — mais la garantie reposait sur le fait que chaque futur chemin y pense. La lecture applique désormais la même règle que la connexion.

`a_activer` reste admis, sans quoi une personne qui vient de recevoir ses accès ne pourrait plus atteindre son propre écran d'activation.

Second facteur

TOTP, par le fournisseur d'authentification. Le niveau d'assurance de la session (`aa11` / `aa12`) est enregistré **par session**, pas par compte : se connecter sur un nouvel appareil ne donne pas d'emblée les droits d'administration.

Il est exigé pour tout acte d'administration. Voir `06-autorisation-et-cloisonnement.md` pour la couche qui l'applique réellement en production.

Un défaut de produit trouvé en route

L'écran d'enrôlement régénère le secret à chaque affichage — c'est voulu : les facteurs non vérifiés sont retirés avant d'en créer un nouveau. Mais sa branche d'erreur n'affichait **ni la clé, ni le formulaire, ni aucun moyen de recommencer**. Une personne qui tombait dessus était bloquée sans recours.

Corrigé : la branche d'erreur porte désormais un bouton « Réessayer ». Ce n'est pas un défaut de sécurité, c'est un défaut d'accès — et l'un empêche l'autre d'être utilisé.

Mots de passe

Vérifiés par le fournisseur d'identité, jamais stockés par AvecStudy. Le changement de mot de passe passe par l'alias technique — le fournisseur ne connaît pas les identifiants locaux.

Un mot de passe temporaire est généré à la remise des accès, avec obligation de changement au premier usage. Tant que cette obligation n'est pas levée, **aucune donnée pédagogique n'est accessible** — vérifié par le test T07 de la batterie RLS. Et un compte non activé ne peut pas lever l'obligation lui-même (T07b).

Journalisation de l'authentification

Ce qui est enregistré en cas d'échec : le compte visé s'il existe, le code d'établissement saisi, la date. **Rien d'autre** — ni le mot de passe, ni l'identifiant saisi, ni l'adresse IP.

Conservation : vingt-quatre heures. Cette durée était déclarée dans la base depuis la migration 0015 et n'était appliquée nulle part (F-09) ; la purge est désormais appelée par la tâche planifiée.

Les 44 appels de journalisation de l'application ont été relus un par un : aucun ne transporte un mot de passe, un secret, un jeton, un cookie ni un contenu.

06 — Autorisation et cloisonnement

Qui voit quoi, et pourquoi on peut le croire.

La règle qui gouverne tout

L'autorisation est appliquée dans la base de données, pas dans l'écran.

Un menu caché ne protège pas une ressource. Une vérification faite dans le composant React protège l'affichage, pas la donnée : il suffit d'appeler l'adresse directement. AvecStudy place donc la décision au plus près de la donnée — dans PostgreSQL, par la sécurité au niveau des lignes (RLS).

Au 23 septembre 2026 : **68 tables, 144 politiques**, et **toutes** les tables des deux schémas portent RLS, `FORCE` compris — y compris pour le propriétaire. Vérifié à chaque recette par `verifier:privileges`.

Les deux schémas

	study	study_prive
Exposé par PostgREST	oui	jamais
Droits accordés à <code>anon</code>	aucun	aucun
Droits accordés à <code>authenticated</code>	par politique	aucun
Ce qu'on y trouve	le travail scolaire, les identités, la structure	sessions, alias techniques, tentatives de connexion, file de travaux

Le cloisonnement est vérifié directement en base, et non déduit de la configuration : `verifier:privileges` interroge `information_schema.role_table_grants` et échoue si `study_prive` accorde quoi que ce soit à `anon`, `authenticated` ou `PUBLIC`.

Les deux clients, et la seule différence qui compte

```
clientUtilisateur(jeton) → applique RLS. C'est la personne qui agit.  
clientExploitation(motif) → contourne RLS. C'est le serveur qui agit.
```

Le second est celui qui peut tout. Il est donc encadré par un mécanisme simple et efficace : **son paramètre motif est un type fermé**. On ne peut pas l'appeler sans déclarer pourquoi, et la liste des motifs admis tient en huit entrées, chacune commentée :

```
demande_commerciale_publicque · amorcage_proprietaire ·  
administration_des_comptes · stockage_des_supports · duplication_de_seance ·  
tache_planifiee · purge_des_tentatives · moderation_des_signalements
```

Un usage abusif se repère en cherchant les appels — `grep clientExploitation` suffit — sans avoir à relire les fonctions. Ajouter un motif est un geste délibéré, visible dans un diff.

Le constat le plus rassurant de cet audit

Une fonction `SECURITY DEFINER` s'exécute avec les droits de son propriétaire. Si une telle fonction accepte **en paramètre** l'identité de la personne pour qui elle agit, et qu'elle est callable depuis un navigateur, alors n'importe qui peut se faire passer pour n'importe qui.

L'audit a cherché exactement ce motif : les 81 fonctions privilégiées des deux schémas, croisées avec celles qui acceptent un paramètre d'identité (`p_profile` , `p_eleve` , `p_acteur` , `p_moderateur` , `p_auteur` ...).

Résultat : toutes sont réservées à `service_role` . Aucune n'est atteignable depuis une session navigateur. Il n'existe donc aucun chemin d'usurpation d'identité depuis un navigateur.

Ce n'est pas un constat ponctuel : `verifier:privileges` le rejoue à chaque recette et échoue si une nouvelle fonction apparaît dans ce cas.

Toutes fixent par ailleurs leur `search_path` , vérifié au même endroit — sans quoi une fonction privilégiée peut être détournée en plaçant un objet homonyme dans un schéma que l'appelant contrôle.

Ce que les 202 tests vérifient

Ils tournent sur **PostgreSQL 17 réel**, en WebAssembly, avec les vraies migrations rejouées depuis zéro à chaque exécution. Ce ne sont pas des simulations : ce sont les politiques du produit, évaluées par le moteur du produit.

Quelques exemples de ce qu'ils établissent :

- un enseignant ne lit pas les copies d'une classe qu'il n'enseigne pas ;
- retirer une affectation **coupe** l'accès de l'enseignant à la séance suivante ;
- un élève ne voit pas un devoir en brouillon ;
- une copie rendue est immuable, et une clé d'idempotence bloque le double clic ;
- une version de contenu scellée ne change plus rétroactivement ;
- l'exploitant voit les deux lycées, **et le travail des élèves lui reste fermé** — il ne peut ni le lire ni y écrire ;
- sans second facteur, un administrateur ne fait rien de privilégié ;
- deux workers ne prennent jamais le même travail ;
- l'alias technique d'un élève ne contient ni nom ni classe.

La leçon la plus coûteuse de cet audit

Tous ces tests cherchaient des **fuites**. Aucun ne vérifiait qu'un chemin **légitime** fonctionnait.

Résultat : en production, un professeur ne pouvait pas ouvrir la copie d'un de ses propres élèves, alors que l'intégralité de la batterie d'isolation passait au vert. Le défaut a été corrigé par la migration 0042, et un test de chemin légitime a été ajouté.

Pire encore : un test échouait à voir le problème parce qu'il encodait l'**attente de la politique** plutôt que le **comportement du produit** — il construisait une ligne dans une forme que le produit ne crée jamais. Un test qui passe pour une mauvaise raison est pire que pas de test, parce qu'on le compte.

C'est de ce constat que vient la règle appliquée depuis : un contrôle doit avoir été **vu rouge** avant d'être cru vert.

Droits d'exécution des fonctions

C'est le sujet du constat F-01, et il mérite d'être compris parce qu'il est contre-intuitif.

PostgreSQL accorde `EXECUTE` à `PUBLIC` sur **toute** fonction créée. Accorder ensuite le droit à `authenticated` **ne retire rien à personne**. Une fonction nouvelle est donc ouverte au monde entier par défaut, et le reste jusqu'à ce que quelqu'un y pense.

Vingt fonctions n'avaient **aucune liste de droits explicite** — donc le défaut de PostgreSQL, donc `EXECUTE` à `PUBLIC`. La migration 0043 révoque tout, accorde explicitement à `service_role`, puis rouvre à `authenticated` — nommément.

Et la liste de ce qu'il faut rouvrir **ne se dresse pas à l'œil** : une politique RLS, un déclencheur, une valeur par défaut s'évaluent avec les droits de celui qui écrit la ligne, et les fonctions `security invoker` propagent l'exigence à ce qu'elles appellent. C'est une fermeture transitive, que `verifier:privileges` calcule désormais à partir de 935 expressions de schéma.

État après application, relevé sur la production le 23 septembre 2026 :

Fonctions exécutables par <code>anon</code>	0
Fonctions ouvertes à <code>authenticated</code> hors registre	0
Fonctions exigées par le schéma et non exécutables	0
Tables portant RLS	68 sur 68
Tables en <code>FORCE</code>	68 sur 68
Fonctions acceptant une identité en paramètre, ouvertes au navigateur	0 sur 46

Le second facteur

Tout acte d'administration l'exige : créer une classe, réinitialiser un accès, suspendre un compte, modérer un signalement, toucher à la facturation.

La garantie est appliquée à deux endroits, et il faut savoir lequel compte vraiment :

- **en base**, par une condition sur le niveau d'assurance de la session — mais la variable de session correspondante n'est pas positionnée par PostgREST en production, si bien que cette couche ne mord que dans les tests ;

- **dans l'application**, par le garde `assuranceSuffisante` — et **c'est celle qui protège réellement la production**.

Cette asymétrie est écrite ici plutôt que passée sous silence : un dossier qui compterait deux couches là où une seule agit compterait faux.

Modération

Le modérateur unique est **l'administrateur d'établissement** — quelqu'un du lycée, pas l'éditeur. C'est un choix : la modération d'un contenu d'élève est un acte éducatif, pas un acte d'exploitation.

Il ne voit pas l'entraide en général — RLS ne la lui montre pas, et c'est voulu. Il voit ce qui a été **signalé**, dans son propre établissement, par un accès borné aux contenus visés, déclaré sous le motif `moderation_des_signalements`. Et il lui faut son second facteur.

07 — Données personnelles

Destiné au délégué à la protection des données. Ce document décrit ce qui est **réellement** dans la base au 23 septembre 2026, relevé colonne par colonne, et non ce que la documentation annonce.

Ce document n'est pas un avis juridique. Il décrit un état technique vérifié. La qualification juridique des traitements, les bases légales retenues et l'annexe de sous-traitance relèvent du DPO de l'établissement et, le cas échéant, d'un juriste.

Qui est quoi

L'**établissement** décide pourquoi et comment les données sont traitées : il choisit d'utiliser AvecStudy, il y inscrit ses élèves, il définit ses classes. Il est **responsable de traitement**.

AvecStudy — Nouh Tifouti, entrepreneur individuel — traite ces données pour le compte de l'établissement et selon ses instructions. Il est **sous-traitant** au sens de l'article 28 du RGPD.

Deux fournisseurs interviennent à leur tour, et sont donc **sous-traitants ultérieurs** : Vercel (hébergement de l'application) et Supabase (base de données, authentification, stockage de fichiers). Voir `annexe-sous-traitance.md` et `08-hebergement-et-transferts.md`.

Ce qui est collecté, relevé dans le schéma

Sur une personne

Donnée	Où	Remarque
Prénom, nom	<code>study.profiles</code>	
Adresse électronique professionnelle	<code>study.profiles</code>	Renseignée pour les personnels ; un élève n'en a pas besoin pour se connecter.
Identifiant local	<code>study.organization_memberships</code> , <code>study_prive.auth_aliases</code>	Remis par l'établissement. Unique dans le lycée.
Rôles, état du compte, état de l'adhésion	<code>study.organization_memberships</code>	
Alias technique	<code>study_prive.auth_aliases</code>	Identifiant opaque vers le fournisseur d'authentification. Un test vérifie qu'il ne contient ni nom ni classe .
Classe, groupe, matière, affectation	<code>class_enrollments</code> , <code>group_memberships</code> , <code>teacher_assignments</code>	

Ce qui n'est pas collecté

Vérifié dans le schéma, pas seulement affirmé :

- **Aucune colonne ne peut porter un INE.** Recherche sur l'ensemble des colonnes des deux schémas : aucun résultat.
- Pas de date de naissance, pas d'adresse postale, pas de numéro de téléphone d'élève ou de famille, pas de photographie.
- Pas de donnée de santé, pas de donnée dite sensible au sens de l'article 9.
- Pas de traceur publicitaire, pas d'outil de mesure d'audience tiers, pas de réseau social intégré.

Les seules coordonnées téléphoniques présentes sont celles d'un contact **commercial** d'établissement (`study.commercial_requests`), saisies par une personne adulte qui demande une démonstration. Ce sont des données professionnelles, pas des données d'élève.

Sur le travail scolaire

C'est le cœur du service, et c'est aussi la catégorie la plus sensible en pratique, même si elle n'est pas « sensible » au sens de l'article 9 : une copie rendue, un commentaire d'enseignant, une note personnelle disent beaucoup d'un adolescent.

Donnée	Où
Copies rendues et leurs versions successives	<code>submissions</code> , <code>submission_versions</code>
Brouillons non rendus	<code>submissions.draft_body</code>
Corrections et appréciations individuelles	<code>lesson_corrections</code> , <code>feedback</code>
Correction commune d'un devoir	<code>assignment_corrections</code>
Contenu des séances et documents déposés	<code>lesson_blocks</code> , <code>content_versions</code> , <code>files</code>
Questions et réponses d'entraide	<code>reponses_entraide</code>
Notes personnelles	<code>personal_notes</code>
Messages	<code>messages</code>
Signalements de contenu	<code>reports</code>

Données techniques

Donnée	Où	Conservation
Sessions ouvertes (empreinte du cookie, appareil, dates)	<code>study_prive.sessions</code>	Expiration par inactivité et expiration absolue ; révocation immédiate à la déconnexion et à toute suspension.
Échecs de connexion (compte visé, code d'établissement saisi, date)	<code>study_prive.tentatives_connexion</code>	24 heures. Voir ci-dessous.
Journal d'audit des actes d'administration	<code>study.audit_events</code>	Conservé. Voir ci-dessous.

Le cookie de session est **opaque** : il ne contient aucune information, pas même chiffrée. C'est une clé vers une ligne de base, rien d'autre. Un cookie volé ne révèle donc rien par lui-même, et se révoque.

Durées de conservation

Il faut distinguer trois cas, parce qu'ils n'ont pas la même solidité.

Appliquée mécaniquement. Les tentatives de connexion : vingt-quatre heures. Cette durée était écrite dans la base depuis la migration 0015 et **n'était appliquée nulle part** — c'est le constat F-09 de cet audit. La purge est maintenant appelée par la tâche planifiée, et son bilan indique le nombre de lignes supprimées. Les sessions expirent, elles, par construction.

Décidée par l'établissement. Le travail scolaire suit l'année scolaire et les décisions du lycée. AvecStudy ne supprime rien de sa propre initiative : ce serait s'arroger une décision qui appartient au responsable de traitement.

Volontairement non purgée. Le journal d'audit. Il enregistre qui a réinitialisé un accès, qui a suspendu un compte, qui a modéré un contenu. Le purger reviendrait à effacer la trace des actes d'administration — c'est-à-dire exactement ce qu'un journal sert à empêcher. Sa croissance est lente (environ 1 190 lignes, 552 ko au 23 septembre 2026) et surveillée.

Point à trancher avec le DPO. Une durée de conservation doit être **déterminée**, y compris pour un journal. « Conservé » n'est pas une durée. Il faut fixer une borne — trois ans, cinq ans, la durée de scolarité plus un an — et l'appliquer. C'est porté aux risques résiduels.

Droits des personnes

L'établissement est l'interlocuteur des élèves et des familles. AvecStudy l'outille.

Droit	Ce qui existe aujourd'hui
Accès	Un élève voit dans son espace l'ensemble de ce qui le concerne : devoirs, remises, corrections, preuves de remise. Un administrateur d'établissement dispose d'un export des accès.
Rectification	Les données d'identité proviennent de l'établissement et se corrigent par lui (écrans d'administration, import de rentrée).
Effacement	Sur décision de l'établissement. Les suppressions passent par les écrans d'administration ; le journal d'audit en garde la trace.
Portabilité	Export au format tableur des accès et des listes. Depuis le 8 octobre 2026 (branche <code>refonte-r2</code> , non déployée) : chaque personne télécharge elle-même, en un geste, ses données au format JSON (« Réglages » Confidentialité », route <code>/app/reglages/export</code>), copies, versions et appréciations publiées comprises. Lectures sous sa session, vérifiées par <code>tests/db/export-personnel.test.mjs</code> .
Opposition, limitation	À traiter par l'établissement au cas par cas.

Point ouvert, à inscrire au plan d'action. L'export complet des données d'un élève n'existe pas sous forme d'une commande unique. Les données sont toutes accessibles et exportables, mais l'opération demande aujourd'hui une intervention de l'éditeur. Pour un délai d'un mois, c'est tenable ; ce n'est pas satisfaisant.

Minimisation, en pratique

Trois choix concrets, vérifiables :

1. **Pas d'INE**, alors que c'est l'identifiant scolaire évident. Il a été écarté parce qu'il suit un élève toute sa scolarité et au-delà : en cas de fuite, il corrèle des bases entre elles. L'identifiant local, lui, ne vaut que dans un lycée et une année.
2. **L'alias technique ne porte ni nom ni classe**. Le fournisseur d'authentification ne sait donc pas qui est qui.
3. **Les élèves n'ont pas d'adresse électronique** dans le système. Le service fonctionne sans, ce qui évite d'en collecter une.

Ce qui est journalisé, et ce qui ne l'est jamais

Les journaux techniques ne contiennent **jamais** : un mot de passe, un secret de second facteur, un jeton, un cookie, le contenu d'une copie, un INE, une URL signée, un nom de fichier, ni le texte d'un commentaire.

Vérifié : les 44 appels de journalisation de l'application ont été relus un par un. Ils écrivent un objet JSON structuré portant un **contexte** et un **code d'erreur**. Aucun ne déverse un objet d'erreur entier — la forme qui, en pratique, recopie tout dans les journaux.

Le journal d'**audit**, lui, est fonctionnel et distinct : il enregistre l'acte d'administration, son auteur et sa cible. C'est une trace voulue, décrite dans `docs/14-journal-audit.md`.

Mineurs

Les utilisateurs sont majoritairement mineurs. Trois conséquences assumées :

- **Aucune publicité, aucun profilage, aucune revente**. Le modèle économique est l'abonnement payé par l'établissement ; les familles ne paient rien et ne sont pas sollicitées.
- **Aucun compte créé par l'élève**. Les accès sont remis par l'établissement ; il n'y a ni inscription libre, ni collecte de consentement à des fins commerciales.
- **Modération**. Les contenus d'entraide peuvent être signalés, et la modération est réservée à l'administrateur d'établissement — quelqu'un du lycée, pas l'éditeur. Voir `docs/17-correction-commune-signalement-nouveautes.md`.

Analyse d'impact (AIPD)

Le traitement porte sur des données de **mineurs**, à **grande échelle** à l'échelle d'un établissement, dans un contexte où les personnes ne peuvent pas s'y soustraire facilement. Ces éléments, pris ensemble, pointent vers la nécessité d'une analyse d'impact.

Recommandation, et elle engage l'établissement. Nous recommandons que le DPO de l'établissement conduise une AIPD avant le déploiement, ou vérifie que le traitement entre dans un cadre déjà couvert par une analyse existante de l'établissement ou du rectorat. AvecStudy fournira toute pièce technique nécessaire. Ce dossier n'en tient pas lieu.

o8 — Hébergement et transferts hors de l'Espace économique européen

Ce qui est établi, et comment

Tout ce qui suit a été **mesuré** le 23 septembre 2026, pas déduit d'un nom de région dans un fichier de configuration.

Où le calcul a lieu

Chaque réponse de Vercel porte un en-tête `X-Vercel-Id` de la forme `<point d'entrée>::<région de calcul>::<identifiant>`. Quand la page est servie depuis le cache, il n'y a pas de région de calcul et le second segment est absent.

Chemin	Avant l'audit	Après correction
<code>/connexion</code>	<code>cdg1::iad1</code> — calcul à Washington	<code>cdg1</code> déclaré
<code>/eleve</code>	<code>cdg1::iad1</code>	<code>cdg1</code> déclaré
<code>/professeur</code>	<code>cdg1::iad1</code>	<code>cdg1</code> déclaré
<code>/</code> , <code>/offre</code> , <code>/mentions-legales</code> , <code>/securite</code>	<code>cdg1</code> seul — page prérendue, aucun calcul	inchangé

Cause : aucune région n'était déclarée. Ni dans `vercel.json`, ni dans le code, ni par un `preferredRegion` sur une route. L'hébergeur appliquait donc son défaut. C'est le constat F-o8, le plus grave de cet audit.

Correction : `vercel.json` déclare `"regions": ["cdg1"]` — Paris. Le contrôle `npm run verifier:hebergement` le vérifie à chaque recette et refuse toute région hors EEE.

Où la base se trouve

Hôte du connecteur : `aws-1-eu-west-1.pooler.supabase.com`. Région AWS `eu-west-1` — Irlande.

Ce que cela change concrètement

Le rendu serveur est l'endroit par où tout passe : il reçoit le cookie de session, le formulaire de connexion, et compose les pages qui portent les noms des élèves et le contenu des copies. Tant qu'il s'exécutait à Washington, ces données faisaient l'aller-retour à chaque affichage, alors même que la base ne quittait pas l'Europe.

C'est corrigé. Mais cela ne suffit pas à répondre à la question que pose un DPO.

Ce qui n'est pas établi — NON VÉRIFIÉ

Nous n'écrirons pas « aucune donnée ne quitte l'Europe ». Cette phrase demande une preuve contractuelle complète, et nous ne l'avons pas.

Une région d'hébergement est un **réglage technique**. Elle dit où une fonction s'exécute et où une base réside. Elle ne dit rien de :

- où les **sauvegardes** sont écrites et répliquées ;
- où les **journaux techniques** du fournisseur sont conservés, et combien de temps ;
- depuis où le **support** du fournisseur peut accéder à l'infrastructure ;
- quels **sous-traitants ultérieurs** chaque fournisseur emploie, et où ;
- quel mécanisme de transfert s'applique — clauses contractuelles types, décision d'adéquation, mesures supplémentaires.

Aucun de ces points ne se lit dans un en-tête HTTP. Tous se lisent dans des documents contractuels.

Pourquoi ces documents ne sont pas résumés ici

Le mandat de cet audit interdit de contacter un tiers, et nous avons choisi de ne pas paraphraser les documents juridiques de Vercel et de Supabase. Deux raisons, et la seconde compte davantage :

1. Ces documents évoluent, et un résumé daté dans un dossier technique se périmé sans prévenir.
2. **Un DPO doit lire le contrat, pas le résumé qu'en fait le sous-traitant.** Un éditeur qui résume l'engagement de son propre fournisseur au bénéfice de son client se place exactement là où il ne faut pas.

Ce qui doit être obtenu, et ce que chaque pièce doit établir

Pièce	Auprès de	Ce qu'elle doit établir
Accord de traitement des données (DPA)	Vercel	Les rôles, les clauses contractuelles types applicables, la liste des sous-traitants ultérieurs, les régions de traitement et de sauvegarde.
Accord de traitement des données (DPA)	Supabase	Idem, plus la localisation des sauvegardes et la politique d'accès du support.
Liste des sous-traitants ultérieurs	les deux	Nom, rôle, pays. Et le mécanisme de notification en cas d'ajout.
Engagement de région	les deux	Que le réglage de région est un engagement , et pas une préférence que l'infrastructure peut outrepasser en cas d'incident.
Politique de notification de violation	les deux	Le délai dans lequel l'éditeur est prévenu, condition de son propre délai de 72 heures.

Tant que ces pièces ne sont pas au dossier, la ligne « transferts hors EEE » porte la mention NON VÉRIFIÉ, et non « conforme ».

Ce que nous pouvons affirmer dès maintenant

- Le calcul est déclaré à Paris et un contrôle automatique le vérifie.
- La base est en Irlande.
- AvecStudy lui-même ne transmet aucune donnée d'élève à un tiers : aucun traceur, aucune mesure d'audience, aucune police de caractères distante — les polices sont auto-hébergées, et la politique de

sécurité du contenu (`connect-src 'self'` , `font-src 'self'` , `img-src 'self'`) interdit techniquement à une page d'appeler un domaine extérieur.

Ce dernier point est vérifiable en une ligne : l'en-tête `Content-Security-Policy` servi par la production n'autorise aucun hôte tiers.

Autres fournisseurs

Aucun autre service ne reçoit de donnée d'élève. Pas d'outil d'analyse, pas de service de messagerie transactionnelle en production, pas de service de surveillance d'erreurs tiers, pas de CDN d'images externe.

C'est un choix, et il a un coût : l'absence de surveillance d'erreurs tierce est aussi une faiblesse, traitée dans `10-journalisation-et-alertes.md`.

Résumé pour le registre

Responsable de traitement	L'établissement
Sous-traitant	Nouh Tifouti, entrepreneur individuel — AvecStudy
Sous-traitants ultérieurs	Vercel (hébergement applicatif), Supabase (base, authentification, stockage)
Région de calcul	<code>cdg1</code> — Paris, déclarée et contrôlée automatiquement
Région de la base	<code>eu-west-1</code> — Irlande
Transferts hors EEE	NON VÉRIFIÉ — dépend des pièces contractuelles listées ci-dessus
Destinataires tiers	Aucun

09 — Sécurité web

Ce que l'application oppose aux attaques classiques du web, éprouvé sur la production le 23 septembre 2026. La grille suit le Top 10 de l'OWASP, sans s'y enfermer.

En-têtes servis par la production

Relevé sur `https://avecstudy.fr/connexion` :

En-tête	Valeur	Ce que cela ferme
Content-Security-Policy	<code>default-src 'self' ; script-src 'self' 'nonce-...' ; 'strict-dynamic' ; object-src 'none' ; base-uri 'self' ; form-action 'self' ; frame-ancestors 'none' ; connect-src 'self' ; font-src 'self' ; upgrade-insecure-requests</code>	L'exécution d'un script injecté, l'exfiltration vers un domaine tiers, la réécriture de la base des URL, le détournement d'un formulaire.
Strict-Transport-Security	<code>max-age=31536000; includeSubDomains</code>	La première requête en clair, pour un an.
X-Frame-Options	<code>DENY</code>	L'encadrement de la page dans un site tiers. Doubé par <code>frame-ancestors 'none'</code> .
X-Content-Type-Options	<code>nosniff</code>	La réinterprétation d'un fichier servi.
Cross-Origin-Opener-Policy	<code>same-origin</code>	Le maintien d'une référence à la fenêtre depuis un autre site.
Cross-Origin-Resource-Policy	<code>same-origin</code>	L'inclusion de ressources depuis un autre site.
Referrer-Policy	<code>strict-origin-when-cross-origin</code>	La fuite d'un chemin complet vers un tiers.
Permissions-Policy	<code>caméra, micro, géolocalisation, paiement, USB et cohortes publicitaires désactivés</code>	L'accès aux capteurs, même par un script légitime.

Le nonce est **régénéré à chaque réponse**. Le détail de la mécanique — et le piège qui a coûté une panne silencieuse en production — est commenté dans `src/proxy.ts` : Next relit la directive `script-src` de l'en-tête **de la requête** pour connaître le nonce, et sans cela, avec `'strict-dynamic'`, plus aucun script de l'application ne se charge.

Limite connue : `style-src 'unsafe-inline'` s'applique à toute l'application ; et douze pages de vitrine prérendues reçoivent `script-src 'unsafe-inline'` faute de pouvoir porter un nonce. Voir F-05 et R-04.

A01 — Contrôle d'accès défaillant

Traité entièrement dans `06-autorisation-et-cloisonnement.md`. L'essentiel : l'autorisation est appliquée **dans la base**, par 144 politiques, et 202 tests la vérifient sur du vrai PostgreSQL. Cacher un menu ne protège pas une ressource, et le proxy ne décide d'aucune autorisation — il ne lit même pas la session.

Ce que l'audit a trouvé sur cet axe : F-01, des fonctions exécutables sans compte. Corrigé.

A02 — Défaillances cryptographiques

- Le cookie de session est **opaque** : une clé vers une ligne de base, sans contenu, même chiffré. Un cookie volé ne révèle rien et se révoque.
- Il porte le préfixe `__Host-`, `HttpOnly`, `Secure`, `SameSite=Lax`, `path=/`. Le préfixe `__Host-` fait que le navigateur **refuse** le cookie s'il n'est pas servi en HTTPS depuis le domaine exact : un sous-domaine compromis ne peut pas en poser un.
- Les jetons du fournisseur d'identité sont chiffrés au repos, avec une version de clé enregistrée à côté — la rotation est donc possible sans casser les sessions vivantes.
- Les comparaisons de secrets se font à temps constant (`timingSafeEqual`), y compris celle du secret de la tâche planifiée : une comparaison naïve fuit la longueur du préfixe correct, et un secret se devine caractère par caractère quand on a la patience.

A03 — Injection

Aucune requête SQL n'est construite par concaténation. L'application parle à la base par PostgREST et par des fonctions à paramètres. Les entrées sont validées par des schémas `zod` avant d'atteindre quoi que ce soit.

Les fonctions de la base fixent toutes leur `search_path`, ce que `verifier:privileges` vérifie à chaque recette : sans cela, une fonction `security_definer` peut être détournée en plaçant un objet homonyme dans un schéma que l'appelant contrôle.

Injection de contenu (XSS) : React échappe par défaut, la CSP à nonce arrête ce qui passerait, et les noms de fichiers affichés sont assainis. Le téléchargement passe par `Content-Disposition: attachment` avec un nom percent-encodé — ce qui ferme aussi l'injection d'en-tête par un nom de fichier contenant un retour à la ligne.

A04 — Conception non sécurisée

Deux choix de conception qui font plus que n'importe quel contrôle :

1. **Le patron BFF**. Le navigateur ne parle jamais à la base. Il parle au serveur d'AvecStudy, qui détient la session et applique les droits. Aucune clé de base n'atteint le navigateur — ce que `verifier:fuites` vérifie sur ce qui est réellement servi.
2. **Deux schémas**. `study` est exposé ; `study_priv` ne l'est jamais et n'accorde aucun droit à `anon` ni à `authenticated`. Sessions, alias techniques et tentatives de connexion y vivent.

A05 — Mauvaise configuration

C'est l'axe sur lequel l'audit a trouvé le plus.

Constat	État
F-08 — aucune région de calcul déclarée, rendu aux États-Unis	corrigé, contrôle permanent
F-01 — <code>EXECUTE</code> jamais retiré à <code>PUBLIC</code>	corrigé et vérifié en production : o fonction exécutable par <code>anon</code>
F-03 / F-04 — deux tables hors de la règle RLS commune	corrigés et vérifiés : 68 tables sur 68 en RLS et FORCE

Et ce qui reste **NON VÉRIFIÉ** : la configuration des deux consoles d'hébergement, faute d'accès. Voir `02-perimetre-et-methode.md`.

A06 — Composants vulnérables

`npm audit` au 23 septembre 2026 :

```
{"info":0,"low":0,"moderate":0,"high":0,"critical":0,"total":0}
```

Aucune vulnérabilité connue, toutes sévérités confondues. Les dépendances sont peu nombreuses et toutes épinglées ; l'inventaire figure dans `package-lock.json`, qui fait office de nomenclature logicielle (SBOM) au format natif de l'écosystème.

A07 — Identification et authentification

Traité dans `05-authentification-et-sessions.md`. Les points saillants :

- **Aucune énumération de comptes.** Un identifiant inconnu et un mot de passe faux produisent le même refus, avec le même vocabulaire — et un identifiant inconnu déclenche quand même une vérification, pour ne pas répondre plus vite. Un compte suspendu produit le même refus encore.
- **Limitation des tentatives** par compte, et désormais détection du balayage d'établissement (F-07).
- **Second facteur** obligatoire pour tout acte d'administration.

A08 — Intégrité des données et du logiciel

- Les copies rendues sont **immuables** : une version scellée ne change plus, et une clé d'idempotence bloque le double clic. Vérifié par les tests T09 et T10.
- `verifier:deploiement` établit que le code servi est bien celui du dépôt.
- Les migrations sont appliquées dans l'ordre et vérifiées ; la batterie RLS reconstruit une base entière à partir de zéro à chaque exécution, ce qui prouve que la suite de migrations est rejouable.

A09 — Journalisation et supervision

La journalisation est propre ; la supervision n'existe pas. Voir `10-journalisation-et-alertes.md` et le risque résiduel R-09.

A10 — Falsification de requête côté serveur (SSRF)

L'application n'effectue aucune requête sortante vers une URL fournie par un utilisateur. Elle ne va chercher que ses propres services. La CSP interdit par ailleurs à une page d'appeler un hôte tiers.

CSRF — traité à part, parce que l'audit y a trouvé quelque chose

Trois défenses, et ce sont bien **celles qui sont déployées** — le module en annonçait une quatrième qui n'était branchée nulle part (F-06).

1. `SameSite=Lax` sur le cookie de session. Une requête POST partie d'un autre site n'emporte pas la session : elle s'exécute anonyme.
2. **La barrière du proxy**, appliquée à toute méthode à effet, avant que la requête n'atteigne quoi que ce soit : Fetch Metadata d'abord, puis `Origin` avec repli sur `Referer`, contre une liste explicite qui ne contient jamais `*`.
3. **La vérification propre à Next** sur les actions serveur, qui compare l'origine à l'hôte.

Les webhooks et la file de travaux sont exemptés de la barrière : ils ne sont jamais appelés depuis une page et s'authentifient par signature ou par secret. C'est écrit dans le proxy, pas implicite.

Six cas rejoués en permanence par `npm run verifier:origine`. Avant correction, deux échouaient.

Dépôt de fichiers

Règle	Comment elle est appliquée
Le type est décidé sur les octets	Signature lue sur les douze premiers octets. Un fichier nommé <code>cours.pdf</code> qui commence par <code><script></code> est un fichier HTML, et il est refusé.
L'en-tête envoyé par le navigateur ne peut jamais faire accepter un fichier	Il ne sert qu'à départager <code>.docx</code> et <code>.pptx</code> , qui partagent la même signature ZIP. Faute de départage, on refuse.
Six formats seulement	PDF, PNG, JPEG, WebP, Word, PowerPoint. Chaque format de plus est un analyseur de plus dans le navigateur des élèves.
20 Mo maximum	Vérifié avant de lire le fichier en mémoire.
Le chemin de stockage est fabriqué côté serveur	Trois identifiants, sans extension, jamais le nom d'origine — qui contiendrait des accents, parfois un nom d'élève, et permettrait de deviner l'adresse d'un autre fichier.
Rien n'est servi en ligne	Route dédiée, <code>Content-Disposition: attachment</code> . Un PDF piégé s'ouvre dans le lecteur de la personne, pas dans l'origine de l'application.
Un fichier n'est servable qu'une fois déclaré tel	La ligne naît en état <code>reserve</code> ; seul le worker peut la déclarer disponible. Vérifié par la batterie RLS.

Ce qui n'est pas fait, et qui est écrit dans le code plutôt que dissimulé : aucun moteur antivirus n'est raccordé. Voir R-06.

Ce qui a été joué sur la production, et rien de plus

L'ensemble des vérifications offensives de ce chapitre tient en **une vingtaine de requêtes HTTP** : six pour la barrière d'origine, une pour la fonction exécutable sans compte, une par page et par script pour la recherche de fuites, quelques-unes pour les en-têtes et les régions.

Aucune action serveur déclenchée, aucune donnée écrite, aucune saturation.

10 — Journalisation et alertes

Ce chapitre est celui où AvecStudy est le plus inégal : la journalisation est propre, la supervision n'existe pas. Les deux sont dites ici avec la même franchise.

Deux journaux, qu'il ne faut pas confondre

Le journal d'audit — fonctionnel, conservé, consultable

`study.audit_events` enregistre les **actes d'administration** : qui a réinitialisé un accès, qui a suspendu un compte, qui a modéré un signalement, qui a créé une classe. L'auteur, la cible, la date, la nature de l'acte.

Il est consultable depuis l'espace d'administration de l'établissement. Il est vérifié à chaque recette (`verifier:journal`) : un acte qui ne laisserait pas de trace ferait échouer la recette.

Il n'est volontairement pas purgé. Effacer la trace des actes d'administration reviendrait à annuler l'objet même d'un journal. Sa croissance est lente — environ 1 190 lignes et 552 ko au 23 septembre 2026 — et surveillée.

Réserve. « Conservé » n'est pas une durée, et le RGPD en demande une **déterminée**. Une borne doit être fixée avec le DPO de l'établissement et appliquée mécaniquement. Porté aux risques résiduels (R-07).

Le journal technique — éphémère, chez l'hébergeur

Les erreurs d'exécution partent sur la sortie d'erreur et sont collectées par l'hébergeur. Elles servent au diagnostic, pas à la traçabilité.

Ce que les journaux techniques ne contiennent jamais

La liste est fermée et elle a été **vérifiée**, pas seulement décidée :

mot de passe · secret de second facteur · jeton · cookie · contenu d'une copie · INE · URL signée · nom de fichier · texte d'un commentaire · adresse électronique · nom d'élève.

Comment cela a été vérifié

Les **44** appels de journalisation de `src/` ont été relus un par un.

- Tous écrivent un objet JSON structuré portant un **contexte** et un **code d'erreur**. Exemple de forme réelle : `{"niveau":"erreur","contexte":"auth.lecture_session","code":"42501"}`.
- **Aucun ne déverse un objet d'erreur entier.** C'est la forme qui, en pratique, recopie tout dans les journaux : `console.error(erreur)` embarque le message, la requête et parfois les paramètres. Les six occurrences ressemblantes sont des clauses `catch`, pas des journalisations.
- Un seul appel touche un sujet sensible — le changement de mot de passe — et il n'écrit qu'un libellé de contexte et un statut HTTP.

Les scripts de recette suivent la même règle dans leur propre sortie : `verifier:fuites` cherche des valeurs de secrets et n'en imprime **jamais** une, pas même tronquée. Il dit le nom de la variable et le fichier. Un contrôle de fuite qui imprime ce qu'il a trouvé recopie la fuite dans les journaux de la recette, et les journaux voyagent plus loin que la page.

Ce qui manque — et c'est le manque le plus structurant après cet audit

Personne n'est prévenu automatiquement de quoi que ce soit.

Concrètement, aucune alerte n'existe pour :

Événement	Ce qui se passe aujourd'hui
Le taux d'erreur monte	Rien. On s'en aperçoit en regardant.
La file de travaux cesse d'avancer	Rien. Un import reste en attente.
Un balayage de connexion est détecté	Le seuil par compte descend — et personne n'est averti .
Une restauration serait nécessaire	Rien.
Un certificat, une clé, une échéance approche	Rien, sauf pour le <code>security.txt</code> , dont l'échéance est contrôlée à la recette.
Le site est indisponible	Rien.

Pourquoi aucun service de surveillance tiers n'est branché

C'est un choix, et il se défend : un service de surveillance d'erreurs reçoit des traces d'exécution, lesquelles contiennent régulièrement des fragments de données — un identifiant, un extrait de requête, parfois davantage. Le brancher reviendrait à ajouter un destinataire aux données d'élèves, et à devoir le déclarer comme sous-traitant ultérieur.

Mais le revers est réel, et il ne doit pas être présenté comme une vertu : un service qui ne prévient personne est un service dont les incidents durent aussi longtemps que le délai avant que quelqu'un regarde. Pour une structure d'une personne (R-01), ce délai peut être un week-end.

Ce qui est recommandé, sans être fait

Par ordre de valeur décroissante, et sans prétendre que c'est en place :

1. **Une sonde de disponibilité externe** sur `/` et sur `/connexion`, qui alerte par message. C'est le moins coûteux et le plus utile.
2. **Une alerte sur le bilan de la tâche planifiée** : elle rend déjà le nombre de travaux traités et de tentatives purgées ; il suffirait de la faire crier quand la file stagne.
3. **Une alerte sur le compteur de balayage**, qui existe désormais en base et qu'il suffirait de lire.
4. Un agrégateur de journaux hébergé en Europe, si la question des sous-traitants est traitée.

Ces quatre points sont le plan d'action de ce chapitre. Ils ne sont pas faits, et ce dossier ne prétend pas qu'ils le sont.

Avancée du 8 octobre 2026 (branche `refonte-r2`, non déployée) : la sonde `/api/v1/etat` existe. Sans secret, elle répond `200 {"etat":"ok"}` si l'application et la base répondent, `503` sinon — de quoi brancher la recommandation 1. Avec `Authorization: Bearer <CRON_SECRET>`, elle ajoute le nombre de travaux en attente depuis plus de 15 minutes et d'échecs sur 24 h, et répond `503` si la file stagne — recommandation 2. **Aucune sonde externe n'est encore branchée** : il faut choisir un service (hébergé en Europe de préférence) et lui confier l'adresse. Tant que ce n'est pas fait, R-09 reste entier.

Traçabilité côté établissement

Ce qu'un établissement peut établir aujourd'hui, sans l'éditeur :

- qui a réinitialisé un accès, quand, et pour quel compte ;
- qui a suspendu ou réactivé un compte ;
- qui a modéré quel signalement ;
- quelles classes et quels comptes ont été créés par un import de rentrée, avec le rapport de l'import.

Ce qu'il ne peut pas établir : les connexions réussies ne sont pas individuellement journalisées de manière consultable. Les sessions actives sont visibles, les échecs récents sont conservés vingt-quatre heures, mais il n'existe pas d'historique « untel s'est connecté tel jour ».

C'est délibéré — un historique de connexion d'élèves est un fichier de présence en puissance, et ce n'est pas ce qu'un outil pédagogique doit produire. Mais si un établissement en avait besoin pour une enquête, il ne l'aurait pas.

11 — Sauvegardes, restauration, continuité, réversibilité

Le chapitre qui répond à la question qu'un chef d'établissement pose en dernier, et qui est souvent la plus importante : et si tout s'arrête ?

Sauvegardes

Les sauvegardes de la base sont assurées par Supabase, au niveau du projet.

NON VÉRIFIÉ. L'audit n'a pas pu établir si la sauvegarde ponctuelle (*point-in-time recovery*) est activée, ni sa profondeur, ni la fréquence des instantanés, ni où les sauvegardes sont écrites. Ces réglages vivent dans la console Supabase, à laquelle l'environnement d'audit n'avait pas accès.

À établir en priorité, et à inscrire dans ce document : — la sauvegarde ponctuelle est-elle active, et sur quelle profondeur ; — à quelle fréquence un instantané complet est pris ; — combien de temps les sauvegardes sont conservées ; — dans quelle région elles sont écrites et répliquées (voir aussi R-02).

Le stockage de fichiers (copies déposées, supports de cours) suit sa propre politique de sauvegarde chez le fournisseur, distincte de celle de la base. Même réserve.

Ce qui existe en propre, et qui ne remplace rien

`npm run sauvegarde` produit une **sauvegarde logique** de la production, écrite pendant l'audit du 23 septembre 2026 parce qu'il fallait pouvoir revenir en arrière après deux migrations, et pouvoir **prouver** qu'on le pourrait.

Elle capture trois choses, et la troisième est la moins évidente :

- **les données** — chaque ligne de chaque table des deux schémas ;
- **les définitions** — le texte exact de chaque fonction, tel que PostgreSQL le rend, ce qui permet de remettre une fonction dans son état d'avant ;
- **l'état des privilèges** — listes de contrôle d'accès, drapeaux RLS, politiques, droits de table. C'est précisément ce qu'un `pg_dump --data-only` ne rendrait pas, et c'est ce que la migration 0043 modifiait.

Un `retour-arriere.sql` en est engendré. Il n'est **jamais exécuté automatiquement** : c'est une pièce à relire.

La vérification (`npm run sauvegarde -- --verifier <dossier>`) recalcule les empreintes SHA-256 du manifeste et confronte les comptes de lignes à la base vivante. Relevé avant migration : **71 fichiers sur 71 intacts, 68 tables au même compte.**

Ce que cela ne remplace pas. Une restauration ponctuelle rejoue le journal des transactions et rend la base à la seconde près. Rien de ce qui précède n'en tient lieu, et la réserve **NON VÉRIFIÉ** ci-dessus reste entière.

Restauration — l'exercice, et sa limite

Une tâche « sauvegarde réussie » ne prouve rien. Le seul fait qui compte est qu'une base restaurée soit exploitable, et cela ne se déduit pas d'un journal de tâche.

AvecStudy outille l'exercice : `npm run restauration:test`, pointé sur une base **restaurée dans un projet séparé**, vérifie que ce qui en sort est cohérent et produit un compte rendu horodaté. Ce qu'il contrôle :

- le schéma est complet — au moins 50 tables dans `study` ;
- le schéma privé existe et reste fermé ;
- **RLS est active partout** — aucune table de `study` sans politique de ligne ;
- les politiques ne se sont pas perdues à la restauration — au moins 90 ;
- les copies remises sont lisibles ;
- aucune inscription orpheline : chaque inscription pointe vers une classe qui existe.

Le troisième point est celui qui justifie l'exercice à lui seul : une restauration qui rendrait les données **sans** leurs politiques de ligne serait pire qu'une perte de données. Le script le regarde explicitement.

Limite, écrite noir sur blanc. Ce script **ne restaure pas** — la restauration dépend de la console de l'hébergeur. Il vérifie une base déjà restaurée. L'exercice complet suppose donc une étape manuelle qui n'a pas été jouée pendant cet audit, faute d'accès à la console, et parce que restaurer par-dessus la production aurait été destructeur.

L'exercice reste à jouer de bout en bout, dans un projet jetable, et son compte rendu à joindre ici.

Continuité de service

Ce qui existe :

- une page de maintenance servie statiquement ;
- des déploiements atomiques : une mise en production qui échoue ne remplace pas la précédente ;
- un retour arrière possible sur le déploiement précédent, immédiat.

Ce qui n'existe pas, et il faut le dire :

- **aucun engagement de disponibilité.** AvecStudy ne promet pas de taux de service, parce qu'il n'a pas les moyens de le tenir contractuellement ;
- **aucune supervision automatique** (voir R-09) : une indisponibilité est constatée quand quelqu'un regarde ;
- **aucune astreinte** (voir R-01).

Pour un usage scolaire, cela veut dire concrètement : une panne un dimanche soir peut durer jusqu'au lundi matin. Un établissement doit en tenir compte dans la place qu'il donne à l'outil — et ne pas en faire le seul canal d'un devoir dont l'échéance tombe un dimanche.

Réversibilité — ce qu'un établissement récupère, et comment

C'est la partie qui doit tenir **sans** l'éditeur, puisque R-01 dit qu'il est une seule personne.

Ce qui est récupérable aujourd'hui, par l'établissement lui-même

Quoi	Comment
La liste des accès (comptes, identifiants locaux, classes)	Export tableur depuis l'espace d'administration
Les classes, groupes, affectations	Écrans d'administration, export
Les documents déposés	Téléchargement depuis les séances

Ce qui demande aujourd'hui l'éditeur

Quoi	Pourquoi
Un export complet des copies et corrections d'une classe ou d'un élève	Il n'existe pas de commande unique (voir R-08)
Un instantané complet de la base	Console de l'hébergeur

Ce qui garantit qu'un tiers pourrait reprendre

- **Le code est intégralement versionné**, sur 93 commits, poussé sur un dépôt distant. Aucune partie du produit ne vit uniquement sur une machine.
- **Le schéma est restructurable à partir de zéro** : la batterie RLS rejoue les 44 migrations à chaque exécution, sur une base neuve. Ce n'est pas une promesse, c'est un fait vérifié plusieurs fois par jour.
- **Les décisions sont écrites dans le code**, à l'endroit où elles s'appliquent, avec leur raison. C'est le seul transfert de connaissance réaliste pour une structure de cette taille, et c'est délibéré.
- **La base est du PostgreSQL standard**. Pas de service propriétaire, pas de format fermé : un instantané `pg_dump` se remonte n'importe où.

Ce qu'un établissement devrait exiger contractuellement

Ce dossier ne peut pas créer une obligation à la place d'un contrat. Ce qu'il peut faire, c'est nommer ce qu'il serait raisonnable d'y inscrire :

1. **Un export complet des données de l'établissement à la demande**, dans un format ouvert, sous un délai fixé.
2. **Un export automatique de fin d'année scolaire**, déposé à l'établissement sans qu'il ait à le demander.
3. **Une clause de réversibilité** : en cas de cessation d'activité de l'éditeur, la remise de l'instantané des données et l'aide à la reprise, dans un délai fixé.
4. **Un séquestre du code**, si l'établissement le juge nécessaire.

Les points 1 et 2 sont réalisables sans difficulté technique et devraient être le plan d'action immédiat de ce chapitre. Les points 3 et 4 relèvent de la négociation contractuelle.

Ce que ce chapitre ne dit pas

Il ne dit pas que les données sont en sécurité en toutes circonstances. Il dit ce qui est outillé, ce qui est vérifié, ce qui reste manuel et ce qui n'a pas été joué. La ligne la plus importante de ce document est celle qui porte la mention **NON VÉRIFIÉ** sur les sauvegardes : tant qu'elle y figure, la question « que se passe-t-il si la base est perdue ? » n'a pas de réponse établie.

12 — Incidents : procédure et exercices sur table

Avertissement liminaire. Ce document ne décrit **aucune équipe qui n'existe pas**. AvecStudy est édité par une seule personne. Il n'y a ni cellule de crise, ni astreinte, ni rotation, ni délai de réponse contractuel. Ce qui suit est la procédure que cette personne applique, et elle est écrite pour que quelqu'un d'autre puisse l'appliquer à sa place.

Qui fait quoi

Rôle	Qui	En pratique
Détection	L'établissement, un utilisateur, l'éditeur, ou un signalement via <code>security.txt</code>	Il n'y a pas de détection automatique. Voir R-09.
Qualification et traitement	L'éditeur	Nouh Tifouti
Notification à l'établissement	L'éditeur	Sans délai, dès qualification
Notification à la CNIL	L'établissement , responsable de traitement	Sous 72 heures, à partir du moment où il a connaissance
Information des personnes	L'établissement	Si le risque est élevé

Le point qui compte, et qui est souvent mal compris : **AvecStudy ne notifie pas la CNIL**. Il est sous-traitant. Son obligation est de prévenir l'établissement « dans les meilleurs délais », ce qui conditionne la capacité de l'établissement à tenir ses 72 heures.

Les six étapes

1. **Consigner.** Date et heure de la découverte, comment on l'a su, ce qu'on a observé — avant toute action, parce qu'agir efface des traces.
2. **Contenir.** Arrêter l'écoulement : révoquer les sessions, changer un secret, retirer un droit, basculer en maintenance. La contention prime sur le diagnostic.
3. **Qualifier.** Quelles données, combien de personnes, quel établissement, quelle fenêtre de temps. Une qualification honnête vaut mieux qu'une qualification rassurante.
4. **Prévenir l'établissement.** Même incomplet. Un premier message qui dit « il s'est passé ceci, je ne sais pas encore l'étendue » est utile ; un message parfait trois jours plus tard ne l'est pas.
5. **Corriger, puis vérifier.** La correction n'est acquise que lorsqu'un contrôle qui échouait passe.
6. **Écrire.** Un compte rendu, et **un contrôle permanent** qui empêche le retour. C'est la règle appliquée aux dix constats de cet audit.

Si un secret est découvert compromis

La procédure est fixe, et l'ordre compte :

1. **Ne pas l'afficher** — ni dans un message, ni dans un ticket, ni dans un compte rendu. Donner son **type** et son **emplacement**.
2. **Le considérer compromis**, même si le doute subsiste. Un secret dont on n'est pas sûr est un secret compromis.
3. **Le faire tourner** immédiatement.
4. **Vérifier les usages** : qui l'utilisait, quels déploiements le portent, quelles machines le détiennent.
5. **Supprimer les traces** : historique de commandes, fichiers temporaires, journaux de recette, variables d'environnement locales.
6. **Retester** : `verifier:fuites` sur ce qui est servi, et un balayage de l'historique Git.

Dix exercices sur table

Joués sur le papier, pas sur la production. L'objet est de savoir à l'avance ce qu'on ferait, parce que la partie coûteuse d'un incident est l'hésitation.

1. Un cookie de session est volé sur un poste du CDI

Ce qui limite déjà : le cookie est opaque — il ne révèle rien — et il expire par inactivité, avec une fenêtre courte sur un poste partagé.

Action : révoquer les sessions du profil (`auth_revoquer_sessions_profil`). Effet à la requête suivante, puisque la session est relue à chaque fois.

À améliorer : un élève ne peut pas révoquer lui-même ses autres sessions depuis son espace. C'est un manque.

2. Un professeur laisse sa session ouverte sur un poste partagé

Action immédiate : l'administrateur suspend puis réactive le compte — les deux chemins de suspension révoquent les sessions.

Prévention existante : une session déclarée « poste partagé » expire beaucoup plus vite. Encore faut-il que la personne l'ait déclarée à la connexion.

3. Un mot de passe d'administrateur est deviné

Ce qui limite déjà : sans second facteur, un administrateur ne fait **rien** de privilégié — vérifié par les tests T16 et T16b. Il verrait ses écrans, il ne pourrait ni créer, ni suspendre, ni réinitialiser, ni modérer.

Action : réinitialiser l'accès, vérifier le journal d'audit sur la période, s'assurer qu'aucun second facteur n'a été enrôlé entre-temps.

4. Un balayage de mots de passe vise un lycée entier

Ce qui limite désormais : au-delà de trente cibles distinctes en quinze minutes, le seuil par compte tombe de cinq à deux (F-07). L'attaquant passe de cinq essais par compte à deux.

Ce qui ne se passe pas : personne n'est prévenu (R-09). C'est le trou le plus net de ce scénario.

Action, si on l'apprend : lire `study.auth_balayage_etablissement(code, 60)` pour mesurer, et décider avec l'établissement d'une réinitialisation groupée si des comptes ont cédé.

5. La clé de service de la base fuite

Gravité maximale : cette clé contourne RLS.

Action : la faire tourner dans la console du fournisseur, redéployer, puis appliquer la procédure « secret compromis » en entier. Vérifier ensuite le journal d'audit et les volumes de tables pour repérer une extraction massive.

Ce qui limite déjà : la clé n'atteint jamais le navigateur — `verifier:fuites` le vérifie sur ce qui est réellement servi, et le balayage des 93 commits n'a trouvé aucun secret.

6. Un professeur dépose par erreur un fichier contenant des données d'un

autre établissement

Action : retirer le fichier, qui passe à l'état non servable ; vérifier dans le journal qui l'a téléchargé ; prévenir les deux établissements concernés.

Ce qui limite : un fichier n'est lisible que par les destinataires du devoir ou de la séance, et jamais servi en ligne.

7. Un élève publie un contenu inapproprié dans l'entraide

Action : signalement par n'importe quel élève ou professeur ; modération par l'administrateur d'établissement, qui est **du lycée** et pas l'éditeur. Trace au journal d'audit.

Ce n'est pas un incident de sécurité — c'est un incident éducatif, et il est traité par l'établissement. C'est exactement pourquoi le modérateur n'est pas l'éditeur.

8. La base est indisponible

Action : bascule en page de maintenance, constat chez le fournisseur, information de l'établissement.

Ce qui manque : personne n'est prévenu automatiquement. Le délai de détection est le délai avant que quelqu'un regarde.

9. Une mise en production casse la connexion

C'est arrivé, et deux fois plutôt qu'une : une politique de sécurité du contenu qui empêchait tous les scripts de se charger, et une révocation de droits qui aurait empêché la création de classes.

Ce qui l'attrape aujourd'hui : la recette complète avant mise en production, et `verifier:deploiement` après. La seconde panne a été prise par la batterie RLS **avant** la production — c'est le cas nominal.

Action si cela passe quand même : retour arrière sur le déploiement précédent, immédiat, puis diagnostic à froid.

10. L'éditeur est indisponible durablement

C'est le scénario que ce dossier ne peut pas résoudre, et il faut le dire plutôt que de l'habiller.

Ce qui existe : le code est intégralement versionné et poussé sur un dépôt distant ; le schéma se reconstruit de zéro ; la base est du PostgreSQL standard, donc reprenable ; les décisions sont écrites dans le code.

Ce qui manque : une clause de réversibilité contractuelle et un export automatique de fin d'année. Ce sont les deux demandes que `11-sauvegardes-et-continuite.md` recommande à l'établissement d'inscrire au contrat.

Ce que ces exercices ont fait apparaître

Trois manques, qui ne sont pas des constats de vulnérabilité mais des trous dans la capacité à réagir :

1. **Aucune alerte automatique**, dans huit scénarios sur dix (R-09).
2. **Un utilisateur ne peut pas révoquer ses propres autres sessions** (scénario 1).
3. **Aucune clause de réversibilité** n'existe aujourd'hui (scénario 10).

Ils sont portés au plan d'action et aux risques résiduels plutôt que corrigés dans l'urgence : les deux premiers demandent du travail produit, le troisième une décision contractuelle qui n'appartient pas à l'éditeur seul.

13 — Constats et corrections

Audit interne du 23 septembre 2026. Dix constats, du plus grave au moins grave. Chacun porte sa preuve, sa correction et le contrôle permanent qui l'empêche de revenir.

Aucun exploit réutilisable ne figure ici, aucun secret, aucune donnée personnelle. Les reproductions décrites sont celles d'un appel HTTP ou d'une requête SQL banale : ce qui compte est le **résultat obtenu**, pas la recette.

État d'application au 23 septembre 2026

Il faut distinguer « corrigé » de « déployé », et ce document ne les confond pas. Une correction écrite et testée qui n'est pas en production ne protège personne.

Les migrations 0043 et 0044 ont été appliquées à la base de production le 23 septembre 2026, après une sauvegarde logique vérifiée (68 tables, 1 343 lignes, 113 définitions de fonctions, l'état complet des privilèges et un retour arrière engendré — 71 fichiers, empreintes SHA-256 recalculées).

Les dix constats sont corrigés et vérifiés en production :

Constat	Contrôle qui l'établit	Preuve
F-01	verifier:rpc-anonyme	Cinq fonctions répondaient HTTP 200 à un appel anonyme ; elles répondent 401 .
F-01	verifier:privileges	« aucune fonction n'est exécutable sans compte »
F-02	verifier:session-suspendue	Joué sur la base réelle : adhésion suspendue et adhésion terminée ne rendent aucun rôle ; un compte actif garde le sien.
F-03	verifier:privileges	tentatives_connexion : RLS activée et forcée.
F-04	verifier:privileges	auth_aliases : RLS activée et forcée.
F-05	verifier:site	Liste exacte, inchangée, justifiée page par page.
F-06	verifier:origine	6 cas sur 6.
F-07	migration 0044 + 4 tests ABUSE-02	auth_balayage_etablissement présente ; seuil adaptatif.
F-08	verifier:hebergement	/connexion , /eleve , /professeur en cdg1 (Paris) ; base en eu-west-1 .
F-09	verifier:purge	La tâche a purgé la ligne de 48 h, gardé celle d'une heure, et applique bien 24 heures.
F-10	verifier:legal	Fichier servi, adresse conforme, valable 365 jours.

Le détail de la fermeture de F-01

Mesurée là où elle se voit : par HTTP, sans aucune session, avec la seule clé publiable que porte n'importe quel navigateur.

Fonction	Avant	Après
remise_reference	200	401
code_de_classe	200	401
normalize_code	200	401
session_mfa_verifiee	200	401
current_user_id	200	401
mes_remisses , preuve_de_remise , mes_nouveautes , eleve_nouveautes , references_de_remisses , devoir_etat , peut_moderer	401 / 404	401 / 404

Cinq, et non une seule comme l'audit l'avait d'abord établi. La plus gênante n'est pas `remise_reference` mais `session_mfa_verifiee` : le prédicat qui décide si un second facteur a été vérifié était interrogeable depuis l'extérieur, sans compte.

Ce que la révocation n'a pas fermé

Une révocation générale peut fermer un écran aussi sûrement qu'elle ferme une faille. Trois contrôles distincts l'excluent :

- `verifier:privileges` : « chaque fonction exigée par le schéma est exécutable » — fermeture transitive calculée sur **935 expressions de schéma**, 19 fonctions dans la fermeture ;
- les **202 tests RLS** rejoués sur une base reconstruite depuis zéro avec les deux migrations ;
- la **recette connectée** jouée sur la production avec des comptes synthétiques, dans les trois espaces.

Ce qui reste NON VÉRIFIÉ

Les consoles Vercel et Supabase, et la séparation des environnements. Aucun identifiant n'était disponible pendant cet audit. Voir `02-perimetre-et-methode.md` pour la liste exacte de ce qu'il faut ouvrir, et le rapport final pour les contrôles manuels à réaliser.

Comment lire la gravité

Niveau	Ce que cela veut dire ici
Élevé	Une promesse faite à l'établissement est fausse, ou une donnée d'élève est traitée autrement qu'annoncé. Aucune autre couche ne rattrape.
Moyen	Une défense manque ou ne couvre pas le cas le plus probable. Une autre couche limite l'effet, mais on ne peut pas s'y reposer.
Faible	Le risque est réel mais borné : une couche voisine tient, ou l'exploitation demande des conditions qu'on ne rencontre pas.
Observation	Rien n'est cassé. Un choix mérite d'être écrit et surveillé, sans quoi il redeviendra une question au prochain audit.

F-o8 — Élevé — Le rendu serveur s'exécutait aux États-Unis

Le constat

La base de données est en Irlande. Le **calcul**, lui, avait lieu à Washington.

```
X-Vercel-Id: cdg1::iad1::...  
      ^^^^  ^^^^  
      Paris  Washington (us-east-1)
```

Le premier segment est le point de présence qui reçoit la requête — Paris. Le second est la région où la **fonction serveur s'est exécutée**. Mesuré le 23 septembre 2026 sur trois chemins rendus à la demande :

Chemin	Région de calcul
/connexion	iad1
/eleve	iad1
/professeur	iad1
/, /offre, /mentions-legales	aucune — pages prérendues servies depuis le cache de cdg1

Base de données : `aws-1-eu-west-1` — Irlande.

Pourquoi c'est le constat le plus important de cet audit

Le rendu serveur est l'endroit où passe **tout**. Il reçoit le cookie de session, le formulaire de connexion avec son mot de passe, et il compose les pages qui portent les noms des élèves, les intitulés de classe, le contenu des copies. Dire à un lycée « la base est en Europe » et ne pas dire « et le traitement, non » n'est pas une imprécision : c'est la partie de la phrase que le lycée voulait entendre.

La cause

Aucune région n'était déclarée. Ni dans `vercel.json`, ni dans le code, ni par un `preferredRegion` sur une route. L'hébergeur a donc appliqué son défaut. Ce n'est pas une erreur de configuration : c'est une configuration absente, et c'est pire, parce qu'il n'y a rien à relire pour s'en apercevoir.

La correction

`vercel.json` déclare `"regions": ["cdg1"]`. Paris est aussi plus proche de la base irlandaise que Washington ne l'était : la latence baisse, ce n'est pas un compromis.

Le contrôle permanent

`npm run verifier:hebergement` relit l'en-tête à chaque recette, sur trois pages rendues à la demande, et refuse toute région hors EEE. Il vérifie aussi que l'hôte de la base nomme une région admise.

Ce contrôle dit **une** chose et il le dit dans son propre commentaire : la région n'a pas rebasculé. Il ne dit pas où les données se trouvent en droit — voir `08-hebergement-et-transferts.md`, et la question ouverte qui y figure.

F-01 — Moyen — EXECUTE n'avait jamais été retiré à PUBLIC

Le constat

PostgreSQL accorde EXECUTE à PUBLIC sur toute fonction créée. Accorder ensuite le droit à authenticated **ne retire rien à personne**. Trente fonctions des schémas study et study_prive restaient donc appelables par un visiteur anonyme, via PostgREST, sans compte.

La plupart sont sauvées par la couche suivante : elles lisent des tables sur lesquelles anon n'a aucun droit, et répondent 401. La défense en profondeur a tenu.

Sauf une. study.remise_reference ne lit aucune table : c'est une empreinte du numéro de version et d'un sel. Un appel anonyme rendait donc une référence d'accusé de remise valable. Reproduit en production : HTTP 200, une référence de la forme R-XXXXXXX .

Pourquoi c'est « moyen » et pas plus

Ce n'est pas une fuite de données. Encore faut-il connaître l'identifiant de version, que RLS protège. C'est l'affaiblissement d'une promesse : la référence est présentée à l'élève comme non devinable, et cessait de l'être pour qui obtiendrait un identifiant par ailleurs.

La correction

Migration 0043 . Un bloc révoque EXECUTE à public et à anon sur **toute** fonction non-déclencheur des deux schémas, accorde explicitement à service_role , puis rouvre à authenticated — nommément, à partir de listes écrites.

Ce que la correction a appris

Elle a cassé la batterie RLS deux fois, et les deux fois pour la même raison profonde : **la liste des fonctions à rouvrir ne se dresse pas à l'œil**.

1. Une politique RLS, un déclencheur, une valeur par défaut, une contrainte s'évaluent avec les droits de **celui qui écrit la ligne**. classes_set_code appelle normalize_code : sans ce droit, un administrateur ne crée plus de classe.
2. normalize_code est security invoker — elle n'apporte donc aucun privilège, et ce qu'elle appelle à son tour, unaccent_fallback , est demandé à l'appelant lui aussi. La chaîne ne s'arrête qu'au premier security definer .

C'est une **fermeture transitive**, pas une liste. verifier:privileges la calcule désormais à chaque recette, à partir de 934 expressions de schéma, et refuse tout écart — dans les deux sens.

Aucune de ces deux pannes n'aurait produit la moindre erreur au déploiement. Elles seraient apparues à la première création de classe de l'année.

F-07 — Moyen — Aucune détection du balayage d'établissement

Le constat

La limitation des tentatives vise le compte : cinq échecs en quinze minutes, puis une temporisation qui double. Le choix de ne pas viser l'adresse IP est **juste**, et il était déjà écrit dans le code — huit cents élèves d'un lycée sortent par la même adresse publique, et les bloquer par l'IP reviendrait à fermer l'établissement à la première erreur de frappe de la salle 204.

Mais un compteur par compte ne voit pas l'attaque la plus probable ici. Un pulvérisateur de mots de passe n'insiste pas sur un compte : il essaie trois mots de passe plausibles — une date de naissance, le nom du lycée, la valeur distribuée à la rentrée — sur huit cents comptes. Chaque compteur reste à trois, sous le seuil de cinq, et **rien ne ralentit jamais**.

Dans un lycée, où les mots de passe sont remis en début d'année et se ressemblent, c'est le scénario qu'il fallait gêner.

Ce qui rend ce constat particulier

La donnée était **déjà collectée**. Le commentaire de `study_prive.tentatives_connexion` dit, depuis la migration 0015, que les tentatives sur un identifiant inconnu sont enregistrées « pour repérer un balayage ». Le code d'établissement est écrit à chaque échec. Rien ne l'a jamais relu. L'intention avait été notée ; la lecture n'avait pas été bâtie.

Aucun test offensif n'a été mené sur ce point : pulvériser des mots de passe, même sur des comptes synthétiques, revient à saturer une API, ce que le mandat interdit. Le constat est **structurel et il se lit dans la signature** : `auth_compter_echecs(p_profile, p_fenetre_minutes)` ne prend que le profil.

La correction

Migration 0044 : `study.auth_balayage_etablissement(p_code, p_fenetre)` compte les **cibles distinctes** en échec — un compte harcelé mille fois ne compte que pour une, c'est l'autre compteur qui s'en occupe.

Au-delà de trente cibles distinctes en quinze minutes, le seuil par compte descend de cinq à deux.

Ce qui a été délibérément écarté

Verrouiller l'établissement. Un verrou déclenché par un tiers est une arme qu'on lui tend : il suffirait d'un balayage volontaire un matin de rentrée pour empêcher un lycée entier de se connecter.

Ici, rien ne ferme. Une personne dont le mot de passe est bon entre, alerte ou pas — c'est ce que vérifie le test `un_balayage_ne_ferme_jamais_l_etablissement_a_QUI_TAPE_JUSTE`. Celle qui se trompe attend trente secondes au lieu de disposer de cinq essais. L'attaquant, lui, tombe de cinq essais par compte à deux.

Le doute penche aussi du bon côté : si la lecture du compteur d'établissement échoue, on suppose qu'il n'y a **pas** de balayage. Supposer l'inverse transformerait une panne technique en gêne pour huit cents personnes, alors que le compteur par compte, lui, continue de protéger chaque compte séparément.

F-06 — Faible — La barrière d'origine se franchissait les mains vides

Le constat

Six requêtes POST vers la production, le 23 septembre 2026. Aucune n'a déclenché d'action serveur : un POST vers l'URL d'une page ne porte pas l'en-tête que Next exige pour exécuter quoi que ce soit. Ce qui est mesuré est **uniquement** le verdict de la barrière, en amont.

Ce que la requête présentait	Réponse	Attendu
Origine propre + <code>Sec-Fetch-Site: same-origin</code>	200	passé
Origine étrangère + <code>cross-site</code>	403	refusée
Origine étrangère, sans Fetch Metadata	403	refusée
Sans <code>Origin</code> , <code>Sec-Fetch-Site: cross-site</code>	403	refusée
Sans <code>Origin</code> ni <code>Sec-Fetch-Site</code>	200	refusée
Sans <code>Origin</code>, <code>Referer</code> étranger	200	refusée

La cause, et c'est elle qui mérite d'être lue

`src/lib/csrf.ts` exportait `verifierMutation`, couverte par sept tests verts. Cette fonction refusait correctement les deux derniers cas.

Personne ne l'appelait. Le proxy portait sa propre copie de la règle, plus indulgente d'un cas, écrite à côté. Les tests d'unité ne pouvaient pas le voir : ils éprouvaient la fonction, pas le chemin.

Le module documentait par ailleurs « trois défenses indépendantes », dont un jeton CSRF à double dépôt. Ce jeton n'a jamais été branché : `NOM_COOKIE_CSRF`, `genererJetonCsrf` et `jetonsCorrespondent` n'étaient référencés que par leurs propres tests. Une défense décrite mais non branchée est pire que son absence, parce qu'on la compte.

Pourquoi « faible »

`SameSite=Lax` prive une requête venue d'un autre site de la session : elle s'exécute anonyme. Exploiter ce cas demanderait un navigateur qui n'envoie ni `Origin` ni `Sec-Fetch-Site` **et** ignore `SameSite` — c'est-à-dire un navigateur d'avant 2020.

La correction

Deux choses plutôt qu'une.

1. La fonction éprouvée est devenue celle que le proxy appelle. Elle refuse désormais une mutation qui ne présente ni origine ni `Referer` utilisable — sauf si le navigateur a lui-même affirmé `same-origin`, ce qu'une page tierce ne peut pas fabriquer, et ce qui évite de casser les envois de formulaire qui n'emportent pas `Origin`.
2. Le jeton a été **retiré** plutôt que laissé en promesse, et le commentaire du module décrit maintenant les trois défenses réellement déployées : `SameSite=Lax`, la barrière du proxy, et la vérification d'origine propre à Next sur les actions serveur.

Le contrôle permanent

`npm run verifier:origine` rejoue les six cas ci-dessus sur ce qui est réellement servi. Il échouait sur deux lignes avant la correction ; il doit maintenant passer sur les six.

F-02 — Faible — Une adhésion suspendue rendait encore ses rôles

`study.auth_lire_session` joignait `organization_memberships` sans filtrer sur l'état. Une session vivante d'un compte suspendu gardait donc ses rôles — donc ses écrans.

Le risque était déjà fermé : les deux chemins de suspension révoquent les sessions du profil. Mais la garantie reposait alors sur le fait que **chaque futur chemin y pense** — un traitement de fin d'année, un import qui désactive. Une règle qu'il faut se rappeler d'appliquer finit par être oubliée.

Correction (migration `0043`) : la lecture de session applique désormais la même règle que la connexion — adhésion `active`, état `actif` ou `a_activer`. `a_activer` reste admis, sans quoi une personne qui vient de recevoir ses accès ne pourrait plus atteindre son propre écran d'activation. La révocation explicite reste en place : celle-ci ne la remplace pas, elle fait que l'oublier ne suffise plus.

F-09 — Faible — Une durée de conservation écrite et jamais appliquée

`study.auth_purger_tentatives()` existe depuis la migration `0015`. Elle supprime les tentatives de connexion de plus de vingt-quatre heures. **Rien ne l'appelait** — ni la tâche planifiée, ni un script, ni la file de travaux.

Relevé en production : trois lignes, toutes datées du 20 septembre, donc au-delà de la durée déclarée.

Le volume est dérisoire et le contenu maigre — un identifiant de compte, un code d'établissement, une date, jamais un mot de passe ni un identifiant saisi. Mais une durée de conservation écrite quelque part et appliquée nulle part est une **ligne fausse dans un registre de traitement**, pas une négligence anodine. C'est la forme de constat qu'un DPO relève en premier.

Correction : la route de la tâche planifiée appelle la purge avant de drainer la file, et rend le nombre de lignes supprimées dans son bilan. Elle est placée là plutôt que dans la file parce qu'elle est unique, brève et idempotente : en faire un travail à programmer ajouterait une pièce qui peut elle-même tomber en panne sans bruit.

F-10 — Faible — Aucun point de contact pour signaler une faille

`https://avecstudy.fr/.well-known/security.txt` répondait 404.

Un référent numérique qui trouve quelque chose n'avait aucune porte normalisée, et surtout aucune phrase lui disant qu'il ne risquait rien à écrire. C'est le genre de manque qui ne produit pas d'incident — il produit des silences.

Correction : un `security.txt` conforme à la RFC 9116, avec les deux adresses de contact, la politique publiée, les langues, et une échéance.

Ce qu'il promet est volontairement modeste, parce qu'AvecStudy est édité par une entreprise individuelle : un accusé de réception sous cinq jours ouvrés, et l'engagement de ne pas poursuivre un signalement de bonne foi. Il n'annonce ni équipe de garde, ni prime, ni délai contractuel — ce serait faux.

Contrôle permanent : `verifier:legal` relit le fichier servi, vérifie que l'adresse correspond à `identite-legale.ts`, et **prévient soixante jours avant l'échéance**. Un point de contact périmé est pire que pas de point de contact.

F-03 et F-04 — Faibles — Deux tables hors de la règle commune

`study_prive.tentatives_connexion` n'avait pas RLS ; `study_prive.auth_aliases` l'avait sans `FORCE` .

Ni l'une ni l'autre n'est atteignable : `study_prive` n'accorde aucun droit à `anon` ni à `authenticated` , ce que `verifier:privileges` vérifie séparément.

Elles sont corrigées quand même (migration `0043`) pour une raison qui n'est pas technique : **une exception se raisonne à chaque audit, alors qu'une règle uniforme se lit une fois**. Le prochain relecteur n'aura pas à refaire le raisonnement.

F-05 — Observation — `unsafe-inline` sur les pages de vitrine

Les douze pages publiques prérendues reçoivent `script-src 'self' 'unsafe-inline'` au lieu de la politique à nonce.

Ce n'est pas un relâchement de confort : une page écrite une fois au build ne peut pas porter un nonce régénéré à chaque requête, et lui en promettre un dans l'en-tête revient à bloquer tous ses scripts.

Le raisonnement qui le rend acceptable est vérifiable page par page : ce que la CSP arrête, c'est l'exécution d'un script **injecté**, et pour qu'un script soit injecté il faut une entrée. Ces douze pages n'en ont aucune — elles n'affichent que du texte écrit dans le dépôt, jamais une donnée venue d'un élève, d'un professeur ou d'un fichier déposé. Tout ce qui rend du contenu saisi par quelqu'un vit dans l'application connectée, rendue à la demande, qui garde la politique stricte.

La liste est **exacte, jamais par préfixe** : une future page privée sous un chemin voisin n'héritera pas de la politique relâchée, et une nouvelle page de vitrine oubliée dans la liste recevra la politique stricte — donc cassera visiblement à la recette.

Ce qui reste à faire : `style-src 'unsafe-inline'` s'applique, lui, à toute l'application. Next injecte des styles en ligne et il n'y a pas d'échappatoire propre aujourd'hui. C'est une limite connue, portée aux risques résiduels (`16-risques-residuels.md`), pas un point réglé.

Récapitulatif

Id	Gravité	Constat	Correction	Contrôle permanent
F-08	Élevé	Rendu serveur exécuté à Washington	<code>vercel.json</code> → <code>cdg1</code>	<code>verifier:hebergement</code>
F-01	Moyen	<code>EXECUTE</code> ouvert à <code>PUBLIC</code> ; référence d'accusé rendue à un anonyme	migration <code>0043</code>	<code>verifier:privileges</code> (fermeture transitive)
F-07	Moyen	Aucune détection du balayage d'établissement	migration <code>0044</code> + seuil adaptatif	4 tests d'unité ABUSE-02
F-06	Faible	Barrière d'origine franchise les mains vides ; fonction testée non branchée	proxy branché sur <code>verifierMutation</code>	<code>verifier:origine</code> (6 cas)
F-02	Faible	Rôles rendus pour une adhésion suspendue	migration <code>0043</code>	batterie RLS
F-09	Faible	Rétention de 24 h déclarée, jamais appliquée	purge dans la tâche planifiée	bilan de la tâche
F-10	Faible	Aucun <code>security.txt</code>	fichier RFC 9116	<code>verifier:legal</code> (+ alerte à 60 jours)
F-03	Faible	<code>tentatives_connexion</code> sans RLS	migration <code>0043</code>	<code>verifier:privileges</code>
F-04	Faible	<code>auth_aliases</code> sans <code>FORCE</code>	migration <code>0043</code>	<code>verifier:privileges</code>
F-05	Observation	<code>unsafe-inline</code> sur douze pages prérendues	liste exacte, justifiée page par page	<code>verifier:site</code>

Ce que cet audit n'a pas pu établir

Trois points sont marqués **NON VÉRIFIÉ**, et le resteront tant que personne ne les aura joués. Ils sont détaillés dans `02-perimetre-et-methode.md`.

- La configuration de la console Vercel (§18) — aucun identifiant n'était disponible dans l'environnement d'audit.
- La configuration de la console Supabase (§19) — l'accès à la base a permis de tout vérifier au niveau SQL, pas les réglages de projet.
- La séparation des environnements (§31) — établir qu'un déploiement d'aperçu ne pointe pas vers la base de production demande de lire les variables d'environnement de la console.

Une table vide ou une migration ancienne ne prouvent pas qu'une protection existe. Ces trois lignes disent « non vérifié » plutôt que « conforme ».

14 — Les contrôles permanents

Destiné au référent numérique. Chaque constat de cet audit a laissé derrière lui un contrôle qui le rejoue. Ce document dit lesquels, ce qu'ils prouvent, et comment les lancer.

Le principe qui gouverne tout ce chapitre

Un test qui n'a jamais échoué ne prouve pas grand-chose.

Cet audit l'a vérifié trois fois, et de manière embarrassante :

- `verifierMutation` était couverte par sept tests verts, et **personne ne l'appelait** (F-06) ;
- le professeur ne pouvait pas ouvrir la copie d'un élève en production alors que tous les tests d'isolation passaient — ils cherchaient tous des fuites, aucun ne vérifiait le chemin légitime ;
- deux variables de couleur CSS n'existaient pas, ce qui ne casse ni la compilation, ni le typage, ni les tests : seulement l'écran.

Tous les contrôles listés ici ont donc été **vus rouges avant d'être vus verts**. Celui qui n'a pas ce parcours est signalé comme tel.

Le contrôle d'ensemble

```
npm run recette:complete
```

Enchaîne la totalité. C'est ce qui doit passer avant toute mise en production.

Contrôles issus de cet audit

```
npm run verifier:privileges
```

Ce qu'il prouve. Qu'aucune fonction de la base n'est exécutable sans compte, que ce qu'une session navigateur peut exécuter correspond exactement à un registre écrit, et — c'est la partie nouvelle et la plus utile — que **le registre couvre tout ce que le schéma exige**.

Ce dernier point est une **fermeture transitive**, calculée à partir de 934 expressions de schéma : politiques RLS, déclencheurs, valeurs par défaut, contraintes, index, vues. Elle suit les appels à travers les fonctions `security invoker` et ne s'arrête qu'au premier `security definir`.

Pourquoi il fallait le calculer et non le lister. La correction du constat F-01 a cassé la batterie RLS deux fois de suite : d'abord `normalize_code`, appelée par trois déclencheurs ; puis `unaccent_fallback`, que `normalize_code` appelle à son tour. Aucune des deux pannes n'aurait produit la moindre erreur au déploiement — elles seraient apparues à la première création de classe de l'année scolaire.

Rouge avant, vert après : oui, deux fois.

Et une troisième fois, contre lui-même. La première exécution complète de la recette après cet audit a révélé que `verifier:privileges` **n'était pas appelé par la batterie**. Il existait, il passait quand on le lançait à la main, et `recette:complete` ne le lançait jamais. C'est précisément le défaut que ce contrôle est fait d'attraper — une protection écrite que rien n'appelle — appliqué à lui-même. Il est désormais câblé dans la section « Contre la base de production ».

```
npm run verifier:rpc-anonyme
```

Ce qu'il prouve. Qu'aucune fonction de la base ne répond à un appel venu d'un navigateur **sans aucune session**, avec la seule clé publiable.

Pourquoi il ne double pas le précédent. `verifier:privileges` lit `has_function_privilege` — les droits tels que PostgreSQL les enregistre. Un attaquant, lui, ne voit pas une table de droits : il voit une adresse HTTP, et entre les deux il y a PostgREST, sa configuration, les schémas qu'il expose et le rôle qu'il endosse sans jeton. Un droit correctement révoqué en base et une fonction malgré tout joignable ne se distingueraient pas dans le premier.

Ce qu'il n'imprime jamais : le corps d'une réponse. Si une fonction répondait, imprimer ce qu'elle rend recopierait la fuite dans les journaux.

Rouge avant, vert après : oui — **cinq** fonctions répondaient 200, et non une seule comme l'audit l'avait d'abord établi.

```
npm run verifier:session-suspendue
```

Ce qu'il prouve. Qu'une session vivante d'un compte suspendu ne rend plus aucun rôle (F-02), et — c'est le cas qui compte le plus — qu'un compte **actif** garde les siens.

Il s'exécute sur la base réelle, dans une transaction annulée : rien ne survit, et il le vérifie avant de rendre la main. Quatre cas, dont le compte « à activer », sans quoi une personne qui vient de recevoir ses accès ne pourrait plus atteindre son propre écran d'activation.

Pourquoi le témoin positif est indispensable. Un contrôle qui ne vérifierait que le refus passerait au vert sur une fonction cassée qui ne rend jamais rien.

Rouge avant, vert après : la batterie RLS l'établissait déjà sur base neuve ; celui-ci vérifie la fonction **réellement déployée**.

```
npm run verifier:purge
```

Ce qu'il prouve. Que la conservation de vingt-quatre heures est **appliquée**, et pas seulement écrite (F-09).

Pourquoi il produit lui-même la situation. Une table vide ne prouve rien : elle ressemble autant à « la purge fonctionne » qu'à « personne ne s'est trompé de mot de passe depuis longtemps ». Le contrôle pose donc une ligne de 48 heures et une d'une heure, appelle la tâche planifiée avec son secret comme l'hébergeur le fait chaque nuit, puis vérifie que la vieille a disparu **et que la récente est toujours là**.

Ce dernier point est le témoin : une purge qui viderait la table entière passerait au vert, et une limitation de tentatives dont l'ardoise s'efface en permanence ne limite plus rien.

Rouge avant, vert après : oui — rien n'appelait la purge.

`npm run sauvegarde`

Ce qu'il produit. Une sauvegarde logique de la production : chaque ligne de chaque table, le texte exact de chaque fonction, et l'état des privilèges — listes de contrôle d'accès, drapeaux RLS, politiques, droits de table. Plus un `retour-arriere.sql` engendré depuis cet état.

Ce qu'il vérifie (`-- --verifier <dossier>`) : les empreintes SHA-256 du manifeste, et la confrontation des comptes de lignes à la base vivante. Un fichier écrit n'est pas une sauvegarde tant que personne n'a relu ce qu'il contient.

Ce qu'il n'est pas, et le script le dit dans son en-tête : une restauration ponctuelle de plateforme. L'état du PITR chez l'hébergeur reste **NON VÉRIFIÉ**.

`npm run verifier:origine`

Ce qu'il prouve. Que la barrière anti-CSRF du proxy se comporte comme la règle le dit, sur ce qui est réellement servi. Six requêtes, six verdicts attendus.

Ce qu'il ne fait pas. Il ne déclenche aucune action serveur : un POST vers l'URL d'une page ne porte pas l'en-tête que Next exige. Ce qui est mesuré est uniquement le verdict de la barrière, ce qui est précisément l'objet.

Rouge avant, vert après : oui — deux cas sur six échouaient.

`npm run verifier:hebergement`

Ce qu'il prouve. Que le rendu serveur s'exécute dans une région de l'EEE, et que l'hôte de la base en nomme une aussi.

Ce qu'il ne prouve pas, et il le dit lui-même dans son en-tête : la localisation **juridique** des données. Un code de région est un nom donné par un fournisseur. Les sauvegardes, les journaux et l'assistance relèvent de documents contractuels — voir `08-hebergement-et-transferts.md` .

Rouge avant, vert après : oui — les trois pages dynamiques étaient en `iad1` .

`npm run verifier:fuites`

Ce qu'il prouve. Qu'aucun secret ne se trouve dans ce que le navigateur reçoit — le HTML, la charge utile React qui y est incluse, et chacun des fichiers de code chargés.

Pourquoi lire ce qui est servi plutôt que le code. Un secret ne fuit presque jamais par une ligne écrite exprès. Il fuit parce qu'un composant serveur a passé un objet entier en accessoire à un composant client, et que l'objet s'est retrouvé sérialisé dans la page. Rien n'échoue, rien ne s'affiche. Ni la compilation, ni le typage, ni les tests d'unité ne voient ce chemin.

Ce qu'il n'imprime jamais : aucune valeur, même partielle, même trouvée. Il dit le nom de la variable et le fichier. Un contrôle de fuite qui imprime ce qu'il a trouvé recopie la fuite dans les journaux de la recette.

Il dit aussi ce qu'il ne peut pas chercher : les variables absentes de l'environnement où il tourne sont nommées dans sa sortie, pour que personne ne prenne le contrôle pour plus large qu'il n'est.

Rouge avant, vert après : non — il est vert depuis sa création. 6 pages et 13 scripts inspectés intégralement, aucun secret.

`npm run verifier:legal`

Existait déjà pour les mentions légales. L'audit y a ajouté le contrôle du **point de contact de sécurité** (`/well-known/security.txt`) : présence, correspondance de l'adresse avec `identite-legale.ts`, et **alerte soixante jours avant l'échéance**.

Un point de contact périmé est pire que pas de point de contact, et la RFC 9116 impose une date d'expiration précisément pour cette raison.

Rouge avant, vert après : oui — le fichier répondait 404.

Contrôles antérieurs, toujours en vigueur

Commande	Ce qu'elle prouve
<code>npm run test:rls</code>	202 tests sur PostgreSQL réel : l'isolation entre établissements, entre classes, entre élèves, et les chemins légitimes.
<code>npm run test:unite</code>	164 tests : authentification, non-divulgence, limitation des tentatives, chiffrement, formats de fichiers, identité légale.
<code>npm run test:navigateur</code>	Parcours publics joués dans un vrai navigateur.
<code>npm run verifier:site</code>	Balises, référencement, en-têtes de sécurité, politique de sécurité du contenu.
<code>npm run verifier:requetes</code>	Les formes de requête que l'application émet.
<code>npm run verifier:stockage</code>	L'état du stockage : aucun objet orphelin, aucun bucket public.
<code>npm run verifier:journal</code>	Le journal d'audit contient bien ce qu'il doit contenir.
<code>npm run verifier:contraste</code>	Contraste WCAG 2.2 AA mesuré sur les pages rendues.
<code>npm run verifier:jurons</code>	Aucune variable de couleur fantôme.
<code>npm run verifier:etat-final</code>	Après recette : il ne reste que le compte de l'éditeur, aucune donnée temporaire.
<code>npm run verifier:deploiement</code>	Le code servi est bien celui du dépôt.
<code>npm run restauration:test</code>	Restauration d'une sauvegarde — voir <code>11-sauvegardes-et-continuite.md</code> .

Ce que les contrôles ne couvrent pas

Il faut le dire aussi clairement que le reste.

- **La console des fournisseurs.** Aucun contrôle automatique ne lit les réglages Vercel ou Supabase : ils demandent des identifiants que l'environnement de recette n'a pas, et qu'il ne devrait pas avoir.
- **La séparation des environnements.** Même raison.
- **PostgREST.** Les tests RLS tournent sur PostgreSQL en WASM, qui n'inclut pas PostgREST. L'isolation est donc vérifiée au niveau du moteur, ce qui est le niveau le plus fort — mais la couche HTTP qui l'expose n'est éprouvée que par les scénarios navigateur. C'est un angle mort connu, porté aux risques résiduels.

- **L'exhaustivité.** 202 tests RLS couvrent les chemins qu'on a su imaginer. Ils ne prouvent pas qu'il n'en existe pas d'autres.

Fréquence

Quand	Quoi
À chaque commit	<code>typecheck</code> , <code>lint</code> , <code>test:unite</code>
Avant chaque mise en production	<code>recette:complete</code> en entier
Après chaque mise en production	<code>verifier:deploiement</code> , puis <code>verifier:origine</code> , <code>verifier:hebergement</code> , <code>verifier:fuites</code> , <code>verifier:legal</code> contre la production
À chaque rentrée scolaire	la recette complète, plus une relecture de ce dossier

15 — Questionnaire d'établissement, rempli

Les questions qu'un référent numérique ou un DPO pose avant de déployer un outil, avec les réponses d'AvecStudy au 23 septembre 2026. Une réponse « non » ou « non vérifié » est aussi utile qu'un « oui » : ce document sert à décider, pas à convaincre.

Identité de l'éditeur

Question	Réponse
Qui édite le service ?	Nouh Tifouti, entrepreneur individuel — micro-entreprise. Nom commercial : AvecStudy.
SIREN / SIRET	979 992 443 / 979 992 443 00023
Adresse	1 avenue d'Alsace, 90000 Belfort, France
Code APE	62.01Z — Programmation informatique
TVA	TVA non applicable, article 293 B du Code général des impôts
Directeur de publication	Nouh Tifouti
Contact	nireo.contacte@gmail.com — 07 81 69 74 77
AvecStudy est-il une société ?	Non. C'est un nom commercial, pas une personne morale. La responsabilité est celle de l'entreprise individuelle.
Combien de personnes travaillent sur le produit ?	Une. Voir R-01, que nous vous demandons de peser en premier.

Données

Question	Réponse
Quelles données d'élèves sont collectées ?	Prénom, nom, identifiant local remis par le lycée, classe, groupes. Et le travail scolaire : copies, corrections, documents, entraide.
L'INE est-il collecté ?	Non. Vérifié : aucune colonne des deux schémas ne peut en porter un.
Date de naissance, adresse, téléphone, photo ?	Non, pour les élèves.
Adresse électronique d'élève ?	Non. Un test de la batterie le vérifie en permanence.
Données de santé ou dites sensibles ?	Non.
Y a-t-il de la publicité ou du profilage ?	Non, et le modèle économique l'exclut : l'abonnement est payé par l'établissement, les familles ne paient rien.
Les données sont-elles revendues ou partagées ?	Non. Aucun destinataire tiers.
Qui est responsable de traitement ?	L'établissement. AvecStudy est sous-traitant (article 28).

Hébergement

Question	Réponse
Où la base est-elle hébergée ?	Supabase, AWS eu-west-1 — Irlande.
Où le traitement a-t-il lieu ?	Vercel, région cdg1 — Paris, déclarée depuis le 23 septembre 2026 . Auparavant, l'hébergeur appliquait son défaut et le rendu s'exécutait à Washington (constat F-08).
Est-ce un hébergement souverain ?	Non. Les deux fournisseurs sont américains. Nous n'écrirons pas « souverain » sans preuve, et il n'y en a pas.
Est-ce un hébergeur de données de santé (HDS) ?	Non, et ce n'est pas requis : aucune donnée de santé n'est traitée.
Des données quittent-elles l'EEE ?	NON VÉRIFIÉ. Les régions sont européennes et un contrôle automatique le vérifie. Les sauvegardes, journaux et accès du support relèvent des contrats des fournisseurs, qui restent à examiner. Voir 08-hebergement-et-transferts.md .
Qui sont les sous-traitants ultérieurs ?	Vercel (hébergement applicatif) et Supabase (base, authentification, stockage). Leurs propres sous-traitants figurent dans leurs contrats, à obtenir.

Sécurité technique

Question	Réponse
Les échanges sont-ils chiffrés ?	Oui, HTTPS obligatoire, HSTS un an avec sous-domaines.
Le cloisonnement entre établissements est-il vérifié ?	Oui. Il est appliqué dans la base , par 144 politiques, et vérifié par 202 tests sur du vrai PostgreSQL à chaque livraison.
Un professeur peut-il voir les copies d'une classe qu'il n'enseigne pas ?	Non. Et retirer une affectation coupe l'accès à la requête suivante.
L'éditeur peut-il lire les copies des élèves ?	Pas par les chemins applicatifs — quatre tests le vérifient. Il détient en revanche les accès à l'infrastructure, comme tout exploitant de base ; cette limite est écrite dans <code>04-modele-de-menace.md</code> .
Le second facteur est-il disponible ?	Oui, TOTP. Il est obligatoire pour tout acte d'administration.
Les mots de passe sont-ils stockés par AvecStudy ?	Non. Ils sont vérifiés par le fournisseur d'authentification.
Y a-t-il une limitation des tentatives ?	Oui, par compte, et depuis cet audit une détection du balayage d'établissement.
Les fichiers déposés sont-ils analysés par un antivirus ?	Non. C'est écrit dans le code plutôt que dissimulé. Le type est en revanche reconnu sur les octets, six formats seulement sont acceptés, et rien n'est servi en ligne. Voir R-06.
Y a-t-il eu un test d'intrusion indépendant ?	Non. Ce dossier est un audit interne . Aucune certification, aucune homologation, aucune qualification.

Exploitation

Question	Réponse
Y a-t-il un engagement de disponibilité ?	Non. Nous ne promettons pas de taux de service, faute de pouvoir le tenir.
Y a-t-il une astreinte ?	Non. Une panne un dimanche soir peut durer jusqu'au lundi.
Y a-t-il une supervision automatique ?	Non. C'est le manque le plus structurant après cet audit (R-09).
Les sauvegardes sont-elles testées ?	Une sauvegarde logique vérifiée est produite et contrôlée par <code>npm run sauvegarde</code> — données, définitions de fonctions, état des privilèges, empreintes SHA-256 recalculées. L'exercice de restauration de bout en bout est outillé (<code>npm run restauration:test</code>) mais reste à jouer , et les réglages de sauvegarde de l'hébergeur (PITR, profondeur, région) sont NON VÉRIFIÉS faute d'accès à la console.
Y a-t-il une certification, un audit indépendant, une conformité attestée ?	Ni l'un, ni l'autre, ni la troisième. Ce sont trois choses distinctes : voir le tableau en tête de <code>02-perimetre-et-methode.md</code> . Ce dossier est un audit interne .
Combien de temps les données sont-elles conservées ?	Le travail scolaire : selon la décision de l'établissement. Les tentatives de connexion : 24 h. Le journal d'audit : conservé, sans durée déterminée — à fixer avec votre DPO (R-07).

Droits des personnes

Question	Réponse
Un élève peut-il accéder à ses données ?	Oui, dans son espace : devoirs, remises, corrections, preuves de remise.
Un export complet des données d'un élève est-il possible ?	Oui, mais pas en un geste : l'opération demande aujourd'hui une intervention de l'éditeur (R-08).
Comment se fait un effacement ?	Sur décision de l'établissement, par les écrans d'administration. Le journal d'audit en garde la trace.
Qui répond aux demandes des familles ?	L'établissement , responsable de traitement. AvecStudy l'outille.

Réversibilité

Question	Réponse
Que récupérons-nous si nous arrêtons ?	Les accès, la structure et les documents s'exportent déjà par vos écrans. Un instantané complet demande l'éditeur.
Et si l'éditeur cesse son activité ?	Le code est intégralement versionné sur un dépôt distant, le schéma se reconstruit de zéro, la base est du PostgreSQL standard. Aucune clause de réversibilité n'existe aujourd'hui — nous recommandons de l'inscrire au contrat. Voir <code>11-sauvegardes-et-continueite.md</code> .

Conformité

Question	Réponse
Une AIPD est-elle nécessaire ?	Le traitement porte sur des mineurs, à grande échelle à l'échelle d'un établissement, sans possibilité réelle de s'y soustraire. Nous recommandons que votre DPO conduise une AIPD , ou vérifie que le traitement entre dans un cadre existant. Ce dossier n'en tient pas lieu.
Un contrat de sous-traitance est-il prêt ?	Un projet d'annexe figure au dossier. Il est marqué « projet à valider par le DPO et/ou un juriste » et n'est pas un avis juridique.
Êtes-vous certifié ISO 27001 ?	Non.
Êtes-vous « conforme ANSSI » ?	Non , et cette expression n'a pas de sens sans qualification formelle. L'audit s'appuie sur les recommandations publiques de l'ANSSI, ce qui est autre chose.
Êtes-vous « certifié RGPD » ?	Non. Aucune certification RGPD n'existe pour ce service.
Avez-vous un point de contact pour signaler une faille ?	Oui, depuis cet audit : https://avecstudy.fr/.well-known/security.txt . Accusé de réception sous cinq jours ouvrés, et engagement de ne pas poursuivre un signalement de bonne foi.

Les trois questions que nous vous invitons à nous poser dans six mois

1. La configuration des consoles Vercel et Supabase a-t-elle été vérifiée, et le second facteur y est-il exigé ? *(R-03)*
2. Les pièces contractuelles sur les transferts hors EEE sont-elles au dossier ? *(R-02)*
3. L'exercice de restauration a-t-il été joué de bout en bout, et son compte rendu joint ? *(chapitre 11)*

Tant que ces trois réponses sont « pas encore », ce dossier reste incomplet, et il le dit.

16 — Risques résiduels

Ce qui reste après les corrections. Rien n'est minimisé ici : un risque résiduel qu'on n'écrit pas est un risque qu'on découvre au mauvais moment.

Nous n'écrirons ni « sécurité totale », ni « inviolable », ni « aucun risque ». Ce document existe parce que ces phrases seraient fausses.

R-01 — L'éditeur est une seule personne

C'est le risque principal, devant tous les risques techniques de ce dossier.

AvecStudy est édité par une entreprise individuelle. Nouh Tifouti écrit le code, exploite l'infrastructure, répond au support et a mené l'audit que vous lisez.

Conséquences, sans atténuation :

- **Pas d'astreinte.** Un incident un dimanche soir attend le lundi matin. Il n'y a pas de rotation, pas de second niveau, pas de délai de réponse contractuel — et annoncer le contraire serait un mensonge vérifiable.
- **Indisponibilité de la personne = indisponibilité du support.** Une maladie, un accident, et il n'y a personne derrière.
- **Pas de séparation des tâches.** Celui qui écrit le code le déploie et détient les accès à la production. Aucune revue par un pair n'est structurelle.
- **L'audit est auto-administré.** Un auditeur qui a écrit le code trouve bien les défauts d'inattention — celui-ci en a trouvé dix — et trouve mal les défauts de conception, parce qu'il les partage.

Ce qui existe en face, et qui est vérifiable :

- le code est intégralement versionné, sur 93 commits, et poussé sur un dépôt distant ;
- la base est restaurable et l'exercice de restauration est outillé ;
- les données sont exportables par l'établissement lui-même ;
- 366 tests automatiques passent avant chaque livraison, et ils constituent une forme de mémoire qui ne dépend pas d'une personne ;
- chaque décision non évidente est écrite dans le code, à l'endroit où elle s'applique, avec la raison — ce qui est le seul transfert de connaissance réaliste pour une structure de cette taille.

Ce que l'établissement doit faire de ce risque : le peser explicitement, comme il pèserait la solidité de n'importe quel fournisseur unique, et prévoir une clause de réversibilité. Voir `11-sauvegardes-et-continuite.md`.

R-02 — Les transferts hors EEE ne sont pas contractuellement établis

Le calcul est désormais déclaré à Paris, la base est en Irlande, et un contrôle automatique le vérifie. Cela ne suffit pas à affirmer qu'aucune donnée ne quitte l'Europe.

Restent à établir sur pièces : la localisation des sauvegardes, celle des journaux des fournisseurs, les accès du support, la liste des sous-traitants ultérieurs et le mécanisme de transfert applicable.

Tant que ces pièces ne sont pas au dossier, la ligne reste NON VÉRIFIÉE. Voir `08-hebergement-et-transferts.md`, qui liste chaque document à obtenir et ce qu'il doit établir.

R-03 — Trois configurations n'ont pas pu être vérifiées

La console Vercel, la console Supabase, et la séparation entre l'environnement d'aperçu et la production. Aucun identifiant n'était disponible dans l'environnement d'audit.

Ce ne sont pas des réserves de style. Tant que personne ne les a ouvertes, on ne sait pas si le second facteur est exigé sur les comptes d'administration de ces deux consoles — ce qui, en cas de compromission, **donnerait accès à tout** : ce sont les seuls accès qui contournent l'intégralité de ce dossier.

C'est aujourd'hui, après la correction des dix constats, le risque **technique** le plus élevé qui reste. Il n'a pas été corrigé parce qu'il n'a pas pu être regardé.

À jouer en priorité, et la liste précise de ce qu'il faut regarder est dans `02-perimetre-et-methode.md` et dans le rapport final.

R-03 bis — La sauvegarde de plateforme n'est pas établie

Même cause, conséquence distincte, et elle mérite sa ligne : **on ne sait pas si la restauration ponctuelle (PITR) est active**, sur quelle profondeur, ni où les sauvegardes sont écrites.

Une sauvegarde logique vérifiée existe désormais (`npm run sauvegarde`, voir le chapitre 11) et elle a servi avant les migrations du 23 septembre 2026. Elle ne remplace pas une restauration ponctuelle : elle est produite à la demande, pas en continu.

Tant que ce point n'est pas établi, la question « que se passe-t-il si la base est perdue ? » n'a pas de réponse.

R-04 — `style-src 'unsafe-inline'` s'applique à toute l'application

La politique de sécurité du contenu est stricte sur les scripts : nonce et `'strict-dynamic'` partout, sauf sur douze pages de vitrine prérendues qui n'affichent aucun texte saisi par quiconque (constat F-05, justifié page par page).

Les **styles**, eux, gardent `'unsafe-inline'` sur l'ensemble du site. Next injecte des styles en ligne et il n'existe pas aujourd'hui d'échappatoire propre.

Ce que cela ouvre : une injection de style permet l'exfiltration de données par sélecteurs d'attribut dans certaines configurations. C'est une attaque étroite, qui suppose déjà une injection — donc déjà un échec de la couche précédente. Mais la couche n'est pas là.

À réévaluer à chaque montée de version majeure de Next.

R-05 — Les tests RLS ne passent pas par PostgREST

Les 202 tests d'isolation tournent sur PostgreSQL 17 en WASM. C'est du vrai PostgreSQL, avec les vraies politiques, et c'est le niveau le plus fort auquel on puisse vérifier une isolation.

Ce qui manque : PostgREST, la couche HTTP qui expose la base. Sa configuration — schémas exposés, rôles, agrégats autorisés, profondeur des jointures — n'est éprouvée que par les scénarios navigateur, qui suivent les chemins légitimes.

Conséquence : une mauvaise configuration de PostgREST qui exposerait un schéma ou un rôle non prévu ne serait pas prise par la batterie. Le cloisonnement des deux schémas est en revanche vérifié directement en base (`verifier:privileges` , section 5) : `study_prive` n'accorde rien à `anon` ni à `authenticated` .

R-06 — Aucun antivirus sur les fichiers déposés

C'est écrit dans le code, à l'endroit où les fichiers sont reçus, plutôt que dissimulé : **aucun moteur antivirus n'est raccordé**, et la colonne `scan_result` le consigne au lieu de laisser croire à une analyse qui n'a pas eu lieu.

Le modèle de confiance retenu : le fichier est déposé par un enseignant identifié de l'établissement et rendu à ses propres élèves.

Ce qui limite le risque, et qui est vérifié :

- le type est reconnu **sur les octets**, jamais sur le nom ou sur l'en-tête envoyé par le navigateur ;
- six formats seulement sont acceptés ;
- le chemin de stockage est fabriqué côté serveur — jamais le nom d'origine ;
- **rien n'est servi en ligne** : tout passe par une route avec `Content-Disposition: attachment` , donc un PDF piégé s'ouvre dans le lecteur de la personne, pas dans l'origine de l'application.

Ce qui reste : un document bureautique piégé transmis d'un poste enseignant compromis vers ses élèves. Le service ne le détecterait pas.

R-07 — Le journal d'audit n'a pas de durée de conservation déterminée

Il n'est volontairement pas purgé : effacer la trace des actes d'administration reviendrait à annuler l'objet même d'un journal. Sa croissance est lente et surveillée.

Mais « conservé » n'est pas une durée, et le RGPD demande une durée **déterminée**. Il faut fixer une borne avec le DPO de l'établissement — durée de scolarité plus un an, trois ans, cinq ans — et l'appliquer mécaniquement, comme la purge des tentatives de connexion l'est désormais.

R-08 — Pas d'export complet des données d'un élève en un geste

Toutes les données d'un élève sont accessibles et exportables, mais rassembler l'ensemble — identité, inscriptions, remises, corrections, notes — demande aujourd'hui une intervention de l'éditeur.

Pour un délai légal d'un mois, c'est tenable. Ce n'est pas satisfaisant, et c'est d'autant moins satisfaisant que R-01 rappelle que cette intervention dépend d'une seule personne.

Réduit le 8 octobre 2026, une fois déployé (branche `refonte-r2`) : chaque personne télécharge elle-même ses données en JSON depuis ses réglages. Reste à la charge de l'éditeur : l'instantané complet d'un **établissement** (clause de réversibilité, `docs/vente/clause-reversibilite.md`).

R-09 — Pas de supervision ni d'alerte automatique

Aucun service de surveillance d'erreurs tiers n'est branché, par choix : cela éviterait d'envoyer des traces d'exécution — qui peuvent contenir des fragments de données — à un destinataire de plus.

Le revers est net : **personne n'est prévenu automatiquement** si le taux d'erreur monte, si la file de travaux cesse d'avancer, ou si un balayage de connexion est détecté. Le nouveau compteur de balayage (F-07) **ralentit** l'attaque ; il ne **prévient** personne.

C'est le manque le plus structurant qui reste après cet audit. Voir `10-journalisation-et-alertes.md`.

8 octobre 2026 : la sonde `/api/v1/etat` existe (branche `refonte-r2`, non déployée). R-09 reste entier tant qu'aucune sonde externe ne l'interroge.

R-10 — La tâche planifiée ne s'exécute qu'une fois par jour

`vercel.json` programme le drain de la file à 3 h du matin. Un travail qui échoue n'est donc réessayé que le lendemain — et c'est aussi la fréquence à laquelle la purge des tentatives de connexion s'exécute.

L'effet sur la sécurité est faible (la fenêtre de rétention devient de fait 24 h + un cycle). L'effet sur le service, lui, est réel : une conversion de document qui échoue le matin attend la nuit.

Ce qui a été corrigé et n'est donc plus ici

Les dix constats F-01 à F-10 sont corrigés et couverts par des contrôles permanents. Voir `13-constats-et-corrections.md` . Ils ne figurent pas dans les risques résiduels, sauf pour la part que ces contrôles ne couvrent pas — R-02 pour la question contractuelle, R-04 pour les styles.

Synthèse

Id	Risque	Portée	Qui peut agir
R-01	Éditeur à une seule personne	Structurelle	L'établissement, par une clause de réversibilité
R-02	Transferts hors EEE non établis	Juridique	DPO + éditeur, sur pièces
R-03	Trois configurations non vérifiées — le risque technique le plus élevé qui reste	Technique	Éditeur, dès l'accès aux consoles
R-03 bis	Sauvegarde de plateforme (PITR) non établie	Technique	Éditeur, dès l'accès à la console Supabase
R-04	<code>style-src 'unsafe-inline'</code>	Technique	Éditeur, à la prochaine version majeure
R-05	PostgREST hors batterie	Technique	Éditeur
R-06	Pas d'antivirus	Assumée	Éditeur, si un établissement l'exige
R-07	Journal sans durée déterminée	Juridique	DPO + éditeur
R-08	Pas d'export complet en un geste	Fonctionnelle	Éditeur
R-09	Pas de supervision	Technique	Éditeur
R-10	Tâche planifiée quotidienne	Fonctionnelle	Éditeur (dépend du plan d'hébergement)

Annexe — Les contrôles manuels qui restent à faire

*Ce que l'audit du 23 septembre 2026 n'a pas pu vérifier, et ce qu'il faut regarder exactement. À cocher, puis à reporter dans le dossier en remplaçant la mention **NON VÉRIFIÉ**.*

Ces points ne sont pas des réserves de style. Les consoles d'administration de nos deux fournisseurs sont **les seuls accès qui contournent l'intégralité de ce dossier** : ni RLS, ni les politiques, ni les contrôles permanents ne s'appliquent à quelqu'un qui s'y connecte. C'est aujourd'hui, après la correction des dix constats, le risque technique le plus élevé qui reste (R-03).

Pourquoi l'audit ne les a pas faits

Aucun identifiant n'était disponible dans l'environnement d'audit : ni jeton Vercel, ni session de CLI, ni répertoire `.vercel`, ni accès à la console Supabase. L'audit disposait d'une chaîne de connexion à la base, ce qui a permis de tout vérifier au niveau SQL — tables, politiques, fonctions, droits, volumes — mais rien au niveau **projet**.

Ouvrir une console demande les identifiants de l'éditeur. Ce n'est pas une opération qu'un audit conduit à la place de la personne concernée.

A — Console Vercel

A1 • Qui a accès, et comment

- ☐ Lister les membres du projet et leur rôle.
- ☐ **Le second facteur est-il exigé sur chaque compte ?** C'est la ligne la plus importante de cette annexe : un accès Vercel sans second facteur donne le déploiement, donc le code servi, donc tout.
- ☐ Vérifier qu'aucun jeton d'accès personnel ancien ne traîne, et révoquer ceux qui ne servent plus.
- ☐ Vérifier les intégrations tierces autorisées sur le projet.

A2 • Variables d'environnement

- ☐ Lister les variables par environnement : production, aperçu, développement.
- ☐ **Vérifier qu'aucune variable de production n'est partagée avec l'environnement d'aperçu** — en particulier `WORKER_DATABASE_URL`, `SUPABASE_SECRET_KEY`, `SESSION_ENCRYPTION_KEY` et `CRON_SECRET`. C'est le point §31, séparation des environnements.
- ☐ Vérifier que les variables sensibles sont marquées comme telles et non exposées au navigateur (aucun préfixe `NEXT_PUBLIC_` sur une valeur privilégiée — l'application le refuse au démarrage, mais la console est l'endroit où l'erreur se commet).
- ☐ Noter la date de dernière rotation de `CRON_SECRET` et de `SESSION_ENCRYPTION_KEY`.

A3 • Déploiements

- [] La **protection des déploiements d'aperçu** est-elle active ? Sans elle, une URL d'aperçu est publique, et un aperçu pointant sur des données réelles serait une fuite.
- [] Vérifier que la région de calcul déclarée (`cdg1`) est bien celle que le projet applique, et qu'aucun réglage de projet ne la contredit.
- [] Vérifier la durée de conservation des journaux de la plateforme.

A4 • Contrat

- [] Obtenir l'**accord de traitement des données (DPA)** et l'annexer au dossier.
 - [] Relever la liste des **sous-traitants ultérieurs** et leurs pays.
 - [] Établir si le réglage de région est un **engagement** ou une préférence que l'infrastructure peut outrepasser en cas d'incident.
 - [] Relever le **délai de notification** en cas de violation : il conditionne le délai de 72 heures de l'établissement.
-

B — Console Supabase

B1 • Accès

- [] **Le second facteur est-il actif sur le compte propriétaire du projet ?** Même remarque qu'en A1, en pire : cet accès donne la base.
- [] Lister les membres de l'organisation et leur rôle.
- [] Relever la date de dernière rotation des clés de service, et la faire tourner si elle est ancienne.

B2 • Sauvegardes — le point le plus urgent

- [] La **restauration ponctuelle (PITR)** est-elle activée ?
- [] Sur quelle **profondeur** ?
- [] À quelle **fréquence** un instantané complet est-il pris, et combien de temps est-il conservé ?
- [] **Dans quelle région** les sauvegardes sont-elles écrites et répliquées ? Une base en Irlande dont les sauvegardes partent ailleurs ne répond pas à la question des transferts (R-02).
- [] Même série de questions pour le **stockage de fichiers**, qui suit sa propre politique.

Tant que ces cases ne sont pas cochées, la question « que se passe-t-il si la base est perdue ? » n'a pas de réponse — et le chapitre 11 le dit.

B3 • Réseau et exposition

- [] Des **restrictions réseau** sont-elles en place sur la base ?
- [] Vérifier la configuration de l'API exposée : quels schémas, quels rôles. L'audit a vérifié le cloisonnement **en base** (`study_prive` n'accorde rien à `anon` ni à `authenticated`) ; la configuration de la couche HTTP qui l'expose, elle, se lit dans la console.

- [] Vérifier qu'aucun bucket de stockage n'est public. (*La recette le vérifie déjà côté base — cette case sert à confirmer que la console dit la même chose.*)

B4 • Contrat

- [] Obtenir le **DPA** et l'annexer.
- [] Relever les **sous-traitants ultérieurs** et leurs pays.
- [] Relever la **politique d'accès du support** : depuis quels pays, sous quelles conditions, avec quelle traçabilité.
- [] Relever le **délai de notification** en cas de violation.

C — L'exercice de restauration, à jouer une fois

- [] Restaurer une sauvegarde dans un **projet jetable**, distinct de la production.
- [] Y pointer `RESTAURE_DATABASE_URL` et lancer `npm run restauration:test`.
- [] Vérifier en particulier la ligne « RLS est active partout » : une restauration qui rendrait les données **sans** leurs politiques de ligne serait pire qu'une perte de données.
- [] Joindre le compte rendu horodaté au chapitre 11.
- [] Supprimer le projet jetable.

D • Ce qu'il faut faire des résultats

Chaque case cochée remplace une mention **NON VÉRIFIÉ** dans le dossier :

Case	Remplace la mention dans
A1, A2, A3	<code>02-perimetre-et-methode.md</code> §18 et §31
A2 (variables par environnement)	<code>16-risques-residuels.md</code> R-03
B1, B3	<code>02-perimetre-et-methode.md</code> §19
B2	<code>11-sauvegardes-et-continuite.md</code> , <code>16-risques-residuels.md</code> R-03 bis
A4, B4	<code>08-hebergement-et-transferts.md</code> , <code>annexe-sous-traitance.md</code> §7
C	<code>11-sauvegardes-et-continuite.md</code>

Si une case révèle un défaut, il prend un numéro à la suite de F-10 et rejoint `13-constats-et-corrections.md` , avec sa preuve, sa correction et — si c'est possible — le contrôle permanent qui l'empêchera de revenir.

Tant que cette annexe porte des cases vides, le dossier est incomplet, et il le dit.

Annexe — Projet d'accord de sous-traitance (article 28 du RGPD)

PROJET — À VALIDER PAR LE DPO ET/OU UN JURISTE

Ce document n'est pas un avis juridique et n'a pas valeur contractuelle en l'état. Il est rédigé par l'éditeur, qui n'est pas juriste, pour servir de point de départ à une discussion. Chaque clause doit être relue, amendée ou écartée par le délégué à la protection des données de l'établissement et, le cas échéant, par un conseil juridique.

Plusieurs champs sont **délibérément laissés ouverts** : ce sont des décisions qui appartiennent au responsable de traitement, pas au sous-traitant.

1. Les parties

Responsable de traitement : l'établissement scolaire, ci-après « l'Établissement ».

Sous-traitant : Nouh Tifouti, entrepreneur individuel, exerçant sous le nom commercial AvecStudy, 1 avenue d'Alsace, 90000 Belfort — SIREN 979 992 443, SIRET 979 992 443 00023. Ci-après « l'Éditeur ».

2. Objet et durée

L'Éditeur traite des données à caractère personnel pour le compte de l'Établissement, aux seules fins de fournir la plateforme pédagogique AvecStudy.

Durée : celle de l'abonnement, augmentée de la période de réversibilité prévue à l'article 10.

3. Nature des traitements

Collecte, enregistrement, organisation, conservation, consultation, modification, extraction et effacement des données décrites à l'article 4, aux fins de :

- gérer les comptes et les accès remis par l'Établissement ;
- permettre le dépôt, la correction et la restitution du travail scolaire ;
- permettre l'entraide entre élèves d'un même groupe, et sa modération par l'Établissement ;
- assurer la sécurité du service et la traçabilité des actes d'administration.

4. Catégories de données et de personnes

Personnes concernées : élèves (majoritairement mineurs), enseignants, personnels d'administration de l'Établissement.

Catégories de données :

Catégorie	Contenu
Identité	Prénom, nom, identifiant local remis par l'Établissement
Coordonnées	Adresse électronique professionnelle (personnels uniquement)
Rattachement scolaire	Classe, groupes, matières, affectations
Travail scolaire	Copies, versions, brouillons, corrections, appréciations, documents, contributions d'entraide, notes personnelles
Données techniques	Sessions, échecs de connexion, journal d'audit

Ne sont pas collectés : l'INE, la date de naissance, l'adresse postale, le téléphone d'un élève ou d'une famille, la photographie, aucune donnée relevant de l'article 9 du RGPD.

5. Instructions

L'Éditeur ne traite les données que sur instruction documentée de l'Établissement. L'utilisation du service, la création des comptes et l'inscription des élèves valent instruction.

L'Éditeur informe l'Établissement s'il estime qu'une instruction constitue une violation du RGPD.

L'Éditeur **ne traite les données à aucune fin propre**. En particulier : pas de publicité, pas de profilage, pas de revente, pas de réutilisation pour l'entraînement d'un modèle, pas de statistique nominative.

6. Confidentialité et sécurité

L'Éditeur est la seule personne ayant accès à l'infrastructure. Il s'engage à la confidentialité.

Les mesures techniques sont décrites dans le présent dossier de sécurité, qui fait partie intégrante de l'annexe :

- cloisonnement appliqué dans la base de données (144 politiques de sécurité de ligne), vérifié par 202 tests automatiques avant chaque livraison ;
- chiffrement des échanges (HTTPS obligatoire, HSTS) ;
- cookie de session opaque, HttpOnly , Secure , préfixe __Host- ;
- second facteur obligatoire pour tout acte d'administration ;
- limitation des tentatives de connexion, par compte et par établissement ;
- journal d'audit des actes d'administration ;
- audit interne documenté et contrôles permanents rejoués à chaque livraison.

À compléter par l'Établissement et l'Éditeur : les mesures que l'Établissement souhaite voir ajoutées, et le calendrier associé. Les manques connus sont listés dans `16-risques-residuels.md` et ne sont pas dissimulés.

7. Sous-traitants ultérieurs

L'Éditeur recourt aux sous-traitants ultérieurs suivants :

Sous-traitant	Rôle	Région déclarée
Vercel	Hébergement de l'application, exécution du rendu serveur	cdg1 — Paris
Supabase	Base de données, authentification, stockage de fichiers	AWS eu-west-1 — Irlande

L'Éditeur informe l'Établissement de tout changement envisagé, et l'Établissement peut s'y opposer.

Point ouvert, à traiter avant signature. La localisation des **sauvegardes**, des **journaux** de ces fournisseurs et des accès de leur **support** n'est pas établie à ce jour. Les accords de traitement de Vercel et de Supabase doivent être obtenus et annexés. Tant qu'ils ne le sont pas, la mention « aucune donnée ne quitte l'EEE » **ne peut pas figurer au contrat**. Voir `08-hebergement-et-transferts.md`.

8. Droits des personnes

L'Établissement répond aux demandes d'exercice de droits. L'Éditeur l'assiste :

- **accès** : chaque personne voit dans son espace ce qui la concerne ;
- **rectification** : par les écrans d'administration de l'Établissement ;
- **effacement** : sur demande de l'Établissement ;
- **portabilité** : export des accès et des listes au format tableur ; export complet des données d'un élève **sur demande à l'Éditeur**, sous un délai à fixer.

À fixer : le délai de réponse de l'Éditeur à une demande d'assistance. Nous proposons **dix jours ouvrés**, ce qui laisse à l'Établissement la marge nécessaire pour tenir son propre délai d'un mois.

9. Violation de données

L'Éditeur notifie l'Établissement **dans les meilleurs délais** après en avoir pris connaissance, et au plus tard **sous 24 heures**, en lui transmettant :

- la nature de la violation, les catégories et le nombre approximatif de personnes concernées ;
- les conséquences probables ;
- les mesures prises ou proposées.

L'Éditeur ne notifie pas la CNIL : cette obligation incombe à l'Établissement, responsable de traitement, sous 72 heures.

À noter, et c'est une limite réelle. Le délai de 24 heures court à partir du moment où l'Éditeur a **connaissance** de la violation. Aucune supervision automatique n'existe aujourd'hui (voir R-09) : une violation peut donc n'être découverte qu'après un délai qui dépend de l'attention humaine. Cette limite doit être connue de l'Établissement avant signature.

10. Sort des données en fin de contrat

Au terme du contrat, au choix de l'Établissement :

- **restitution** des données dans un format ouvert et exploitable, puis suppression chez l'Éditeur et ses sous-traitants ultérieurs ;
- ou **suppression** directe, avec attestation.

À fixer : le délai de restitution, le délai de suppression, le format exact, et la durée pendant laquelle les sauvegardes des fournisseurs peuvent encore contenir des données après suppression — ce dernier point dépend des contrats à obtenir (article 7).

Clause recommandée par l'Éditeur lui-même, et que nous invitons l'Établissement à exiger : une **clause de réversibilité en cas de cessation d'activité**. AvecStudy est édité par une seule personne ; c'est le risque R-01 du dossier de sécurité, et il serait malhonnête de faire signer cette annexe sans le nommer ici. Nous proposons que le contrat prévoie la remise d'un instantané complet des données de l'Établissement dans un délai fixé, et un export automatique de fin d'année scolaire.

11. Audit

L'Établissement peut demander à l'Éditeur toute information nécessaire pour démontrer le respect de l'article 28, et faire réaliser un audit.

L'Éditeur met à disposition, sans demande préalable : le présent dossier de sécurité, les résultats des contrôles permanents, et la liste des constats d'audit avec leur état de correction.

Précision d'honnêteté. Le dossier de sécurité joint est un audit **interne**, mené par l'Éditeur sur son propre produit. Il n'a fait l'objet d'aucune revue indépendante, d'aucune certification et d'aucune homologation. L'Établissement reste libre de faire réaliser un audit par un tiers à ses frais, et l'Éditeur s'engage à y coopérer.

Ce que ce projet ne règle pas

Pour que la discussion parte du bon endroit, voici ce qui reste ouvert :

1. Les accords de traitement de Vercel et Supabase, à obtenir et annexer.
2. La durée de conservation du journal d'audit, à déterminer.
3. Les délais de réponse, de restitution et de suppression, à fixer.
4. La clause de réversibilité en cas de cessation d'activité.
5. La nécessité d'une analyse d'impact (AIPD), à trancher par le DPO.

Rappel final : PROJET — À VALIDER PAR LE DPO ET/OU UN JURISTE.